

First-Press Quiz Game

Internet of Things (IoT)

First Semester – Academic Year 2026–2027

Team Members

Name	Student ID
Maisoon Mansoor Mohameed AL-Matari	2023015305
Bashayer Mansoor Mohameed AL-Matari	2023015606
Zainab Mansoor Mohameed AL-Matari	2023015604
Alia’a Abdullah Mohameed AL-Madeni	2024017239
Shahad Ali Mohameed AL-Snbani	2023016056

Instructor Name:

Eng. Majd AL-Sorihi

University Name:

Al-Razi University

Submission Date:

16 September 2026

2. Abstract

This project presents the design and implementation of an IoT-based three-player quiz game that combines physical hardware with a real-time software system. The project addresses the need to accurately identify the first player who presses a button, prevent other players from responding during the same round, evaluate answers, and manage player scores automatically. The system is built around Node-RED, which receives player events from an ESP32, processes the first-player selection, controls the round-lock mechanism, manages questions and answers, stores player scores, and handles round and score resets. MQTT is used for communication between the ESP32 and Node-RED through a Mosquitto broker, while RESTful HTTP APIs are implemented to retrieve scores and reset game rounds and support integration with the Flutter application. The system also uses Node-RED Dashboard 2.0 to display quiz questions, answer results, the first player who pressed the button, the status of the current round, and the scores of all three players, in addition to providing controls for resetting rounds and scores. The hardware consists of an ESP32, three push buttons, three LEDs with 220-ohm resistors, a buzzer, a P817 optocoupler, a breadboard, and connecting wires. The completed system integrates physical player interaction, MQTT communication, Node-RED processing, RESTful APIs, Dashboard 2.0, and Flutter into a unified interactive quiz-game architecture.

Keywords: IoT, Node-RED, ESP32, MQTT, RESTful API, Dashboard 2.0

3. Introduction

The Internet of Things (IoT) provides an effective way to connect physical devices with software systems and enable real-time interaction. This project applies IoT concepts to develop an interactive quiz game for three players.

The system combines physical hardware, including an ESP32, push buttons, LEDs, and a buzzer, with Node-RED, MQTT, RESTful APIs, Dashboard 2.0, and a Flutter application. These components work together to detect player actions, manage questions and scores, and display the game status.

The following sections describe the motivation for the project, the problem being addressed, the main project objectives, and the scope and boundary limits of the proposed system.

3.1 Background and Engineering Motivation

The Internet of Things (IoT) enables physical devices to communicate with software systems in real time. This project applies IoT concepts to a three-player quiz game using an ESP32, physical buttons, MQTT, Node-RED, Dashboard 2.0, RESTful APIs, and

Flutter. Node-RED was selected because its event-driven, node-based environment simplifies system integration, testing, and rapid development.

3.2 Problem Statement

The project addresses the need for a fair and automated quiz system that can accurately identify the first player to press a button. The system must prevent other players from responding during the same round, process answers, manage scores, and reset the game state for the next question.

3.3 Project Objectives

The main objectives are to:

- Develop an IoT quiz game for three players.
- Connect the ESP32 with Node-RED using MQTT.
- Detect and lock the first player who presses a button.
- Manage questions, answers, and player scores.
- Display the game status using Dashboard 2.0.
- Provide RESTful APIs for Flutter integration.
- Test the system using MQTTX, Postman, and Node-RED Debug.

3.4 Scope and Boundary Limits

In Scope: The project includes the ESP32, three push buttons, three LEDs, resistors, buzzer, P817 optocoupler, breadboard, Arduino, MQTT/Mosquitto, Node-RED, Dashboard 2.0, RESTful APIs, MQTTX, Postman, and Flutter.

Out of Scope: ThingSpeak, Telegram, WhatsApp, and Email integrations were excluded from the project requirements. Therefore, they are not implemented in the final system.

4. System Architecture & High-Level Overview

This section presents the complete architecture of the IoT-based three-player quiz game. The system combines physical hardware, MQTT communication, Node-RED processing, Dashboard 2.0, RESTful APIs, and a Flutter application to provide real-time interaction between the players and the quiz system.

4.1 End-to-End System Concept

The system consists of three main stages:

1. Data Acquisition / Input

The main physical input comes from three push buttons connected to the ESP32, with one button assigned to each player. Each player also has a dedicated LED. The hardware additionally includes 220-ohm resistors, a buzzer, a P817 optocoupler, a breadboard, and connecting wires.

When a player presses a button, the ESP32 detects the event. The first valid player event is communicated to the software system using the MQTT protocol through the Mosquitto broker.

The Node-RED flow subscribes to the MQTT topic:

quiz/winner

The received payload identifies the player as 1, 2, or 3.

Dashboard buttons and Inject nodes are also used as software inputs during testing.

2. Processing in Node-RED

Node-RED is the main processing and integration layer of the system.

After receiving the ESP32 player number, the Convert ESP32 Player Function node converts the numeric value into P1, P2, or P3.

The First Player Lock Function node then checks the roundLocked Flow Context variable. If no player has already been accepted, the first player is selected and roundLocked is set to true. Any later player event in the same round is ignored.

Node-RED also manages the quiz questions, checks the selected answers, stores the scores of P1, P2, and P3, resets rounds, resets scores, and provides RESTful API endpoints.

The complete Node-RED implementation is contained in the main flow tab named Flow 15.

3. Delivery / Output

The processed information is delivered to Dashboard 2.0, where the user can view the current question, answer result, first player, round status, and the scores of the three players.

Node-RED also publishes a reset command to the ESP32 through:

quiz/reset

This allows the physical hardware to be prepared for a new round.

RESTful HTTP endpoints are used to expose game information and control functions. These endpoints can be tested using Postman and support integration with the Flutter application.

4.2 High-Level Block Diagram — REQUIRED

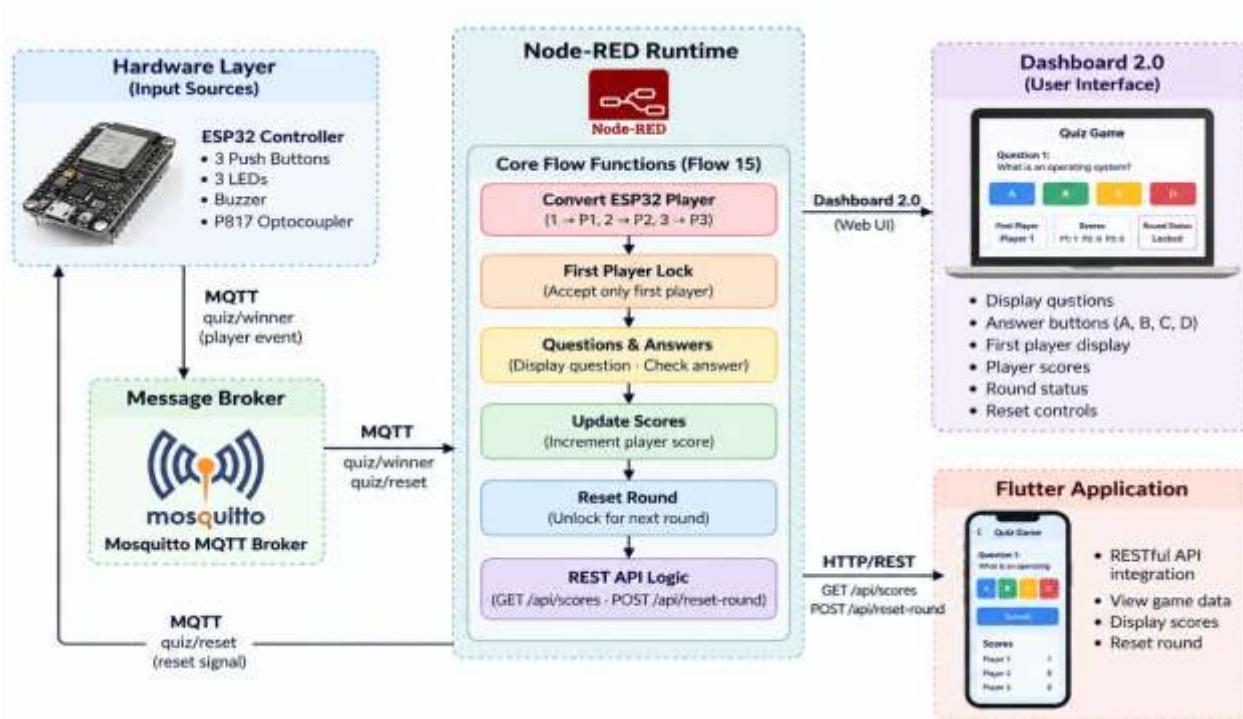


Figure 1. High-Level Architecture of the IoT-Based Quiz Game

4.3 Operating Scenarios and Trigger Events

Scenario 1: First Player Selection

When a question is displayed, all three players are allowed to press their physical buttons. The ESP32 detects the first valid button press and sends the player number to Node-RED through MQTT. Node-RED identifies the player and sets `roundLocked` to true. Therefore, additional button presses from the other players are ignored during the same round.

Scenario 2: Answer Processing

After the first player is selected, the player answers the displayed question. Node-RED compares the selected answer with the correct answer stored in `flow.correctAnswer`. If

both values match, the system displays a correct-answer message; otherwise, it displays an incorrect-answer message.

Scenario 3: Round Reset

When a new round is started, Node-RED changes roundLocked to false. A reset message is also published through the quiz/reset MQTT topic so that the ESP32 and physical hardware can return to the ready state.

Trigger Events Table

Trigger Event	Condition	System Response
Player presses button	First valid press and roundLocked = false	Accept the player, activate the corresponding game path, and set roundLocked = true
Another player presses	roundLocked = true	Ignore the additional player event
Player selects correct answer	Selected answer = correctAnswer	Display the correct-answer message
Player selects wrong answer	Selected answer $\neq$ correctAnswer	Display the incorrect-answer message
Reset Round button/API	Reset command received	Set roundLocked = false and send reset command to ESP32
Reset Scores button	Score reset command received	Set P1, P2, and P3 scores to zero

5. Node-RED Flow Architecture & Data Design

The Node-RED flow is the main processing layer of the quiz game. It connects the ESP32, MQTT communication, game logic, Dashboard 2.0, and RESTful APIs. The flow manages player selection, round locking, questions, answers, scores, and reset operations.

5.1 Flow Decomposition

The project uses one main Node-RED flow named **Flow 15**.

The flow is divided logically into the following functions:

Flow Tab / Subflow Name	Purpose	Key Nodes Used	Input Data	Output Data
Flow 15	Controls and integrates the complete three-player quiz game	MQTT In, MQTT Out, Inject, Switch, Function, Change, HTTP In, HTTP Response, Debug, ui-button, ui-text	ESP32 player number, player test messages, answer selections, HTTP requests, reset commands	Player identity, scores, questions, answer results, round status, HTTP JSON responses, MQTT reset command

Data Flow Between Components

The data begins either from the ESP32 through MQTT or from testing/dashboard input nodes. ESP32 player events are received by the ESP32 Winner MQTT In node and passed to Convert ESP32 Player.

The converted player identifier is then processed by First Player Lock, which ensures that only one player is accepted during each round. The selected player's information is passed to the display and score-management section.

The question section generates the current quiz question, while the A, B, C, and D Dashboard buttons send answers to Check Answer.

The REST API section operates within the same flow and provides access to current scores and round-reset functionality.

The project uses one main flow rather than multiple tabs. This keeps the prototype in one workspace and makes the complete game sequence visible during development and demonstration. For a larger implementation, these functions could be separated into reusable subflows.

5.2 Flow Diagrams — REQUIRED (Screenshot)

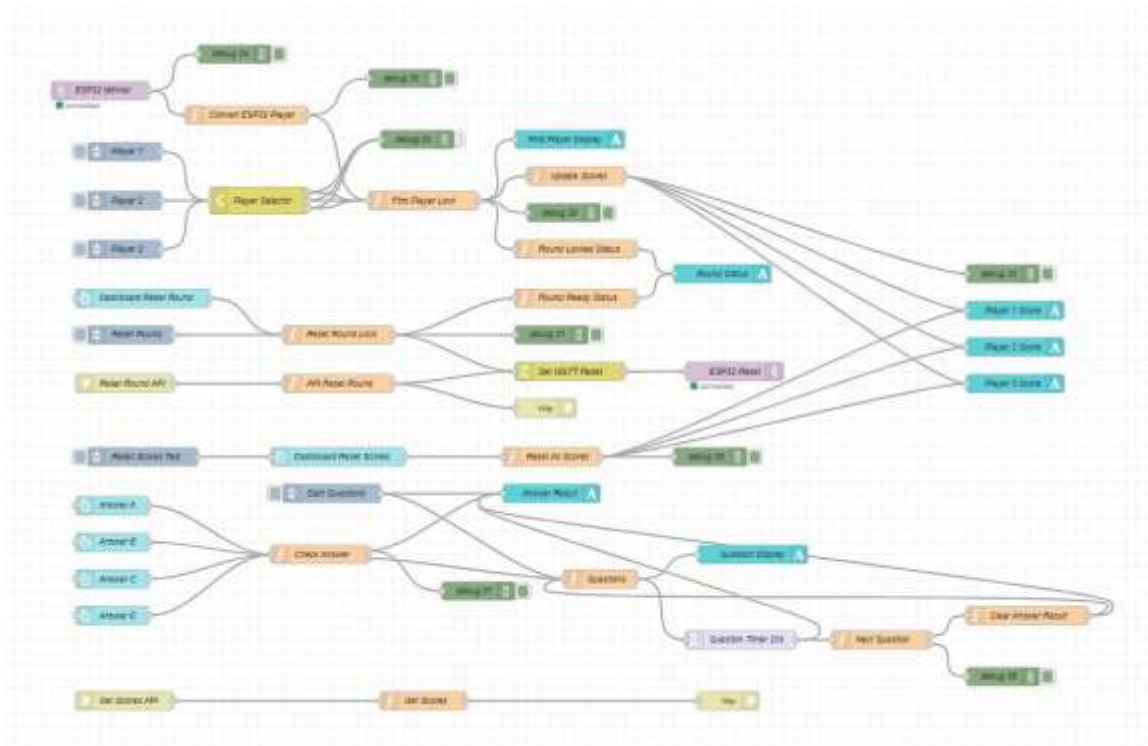


Figure 2. Complete Node-RED Flow of the IoT-Based Quiz Game.

The flow contains the complete implementation of MQTT communication, player conversion, first-player locking, score management, questions and answers, round resetting, RESTful APIs, Dashboard 2.0 widgets, and debugging nodes.

5.3 Msg Payload Schemas

Node-RED uses `msg.payload` to transfer data between nodes. Different payload formats are used depending on the processing stage.

Stage 1: Output of MQTT In Node

```
{"topic": "quiz/winner", "payload": 1, "qos": 0, "retain": false}
```

Field	JSON Type	Description	Format / Example
topic	String	MQTT topic used for winner event	quiz/winner
payload	Number / String	Player identifier received from ESP32	1, 2, or 3

Field	JSON Type	Description	Format / Example
qos	Number	MQTT Quality of Service	0
retain	Boolean	MQTT retained-message state	false

Stage 2: After Convert ESP32 Player

```
{"payload": "P1"}
```

Field	JSON Type	Description	Format / Example
payload	String	Internal player identifier	P1, P2, P3

Stage 3: After First Player Lock

```
{"payload": "Player 1"}
```

Field	JSON Type	Description	Format / Example
payload	String	Human-readable accepted player	Player 1

Stage 4: Score Data

```
{"P1": 0, "P2": 0, "P3": 0}
```

Field	JSON Type	Description	Format / Example
P1	Number	Player 1 score	0, 1, 2...
P2	Number	Player 2 score	0, 1, 2...
P3	Number	Player 3 score	0, 1, 2...

Stage 5: REST API Output

```
{"success": true, "scores": { "P1": 0, "P2": 0, "P3": 0}, "roundLocked": false}
```

Field	JSON Type	Description	Format / Example
success	Boolean	Indicates successful API processing	true
scores	Object	Contains all player scores	{"P1":0,"P2":0,"P3":0}
roundLocked	Boolean	Current round lock state	true / false

Message Conventions

msg.topic naming convention:

MQTT topics use the structure:

quiz/<function>

Examples:

quiz/winner

quiz/reset

Timestamp format:

No custom timestamp is currently added to the quiz messages.

Units convention:

No physical measurement units are required because the main values represent player identifiers, answers, and scores.

Custom message properties:

The main project state is stored using Flow Context variables:

roundLocked

scores

questionIndex

correctAnswer

timeLeft

5.4 Function Nodes & Logic

The Node-RED flow uses several Function nodes to process player data, control the first-player locking mechanism, manage scores, reset rounds, and provide RESTful API responses.

Function node: Convert ESP32 Player

- **Purpose:** Convert the numeric player identifier received from the ESP32 into the internal player format used by Node-RED.
- **Input contract:** {"payload": 1}
- **Output contract:** {"payload": P1}
- **State tracking:** No Flow or Global Context is used.

```
let p = Number(msg.payload);  
if (p === 1) { msg.payload = "P1"; } else if (p === 2) { msg.payload = "P2"; }  
else if (p === 3) { msg.payload = "P3"; } else { return null; }  
return msg;
```

- **Algorithm explanation:** The incoming payload is converted to a number. Values 1, 2, and 3 are mapped to P1, P2, and P3. Any other value is considered invalid and is discarded using return null.

Function node: First Player Lock

- **Purpose:** Accept only the first player who presses a button during the current round.
- **Input contract:** {"payload": P1}
- **Output contract:** {"payload": "player 1"}
- **State tracking:** Uses:
flow.roundLocked
to store whether a player has already been accepted.

```
let locked = flow.get("roundLocked") || false;
if (locked === true) { return null;}
flow.set("roundLocked", true);
if (msg.payload === "P1") {
  msg.payload = "Player 1";}
else if (msg.payload === "P2") {
  msg.payload = "Player 2";}
else if (msg.payload === "P3") {msg.payload = "Player 3";}
else { return null;}
return msg;
```

- **Algorithm explanation:** The function first reads roundLocked. If it is already true, the incoming message is discarded. Otherwise, the function locks the round and converts the internal player identifier into a readable player name.

Function node: Update Scores

- **Purpose:** Update and store the score of the selected player.
- **Input contract:** {"payload": "player 1"}
- **Output contract:** Produces updated score values for the three players.
{"payload": {"P1": 1,"P2": 0,"P3": 0 }}
- **State tracking:** flow.scores
stores the current scores.
let scores = flow.get("scores") || {P1: 0,P2: 0,P3: 0};
if (msg.payload === "Player 1") {scores.P1++;}
else if (msg.payload === "Player 2") {scores.P2++;}
else if (msg.payload === "Player 3") {scores.P3++;}
flow.set("scores", scores);

```
msg.payload = scores;
return msg;
```

- **Algorithm explanation:** The function retrieves the current scores or creates an initial zero-score object. It then increments the corresponding player value, saves the updated object in Flow Context, and outputs the scores.

Function node: Reset Round Lock

- **Purpose:** Prepare the game for a new round.
- **Input contract:** Receives a reset event from the connected reset node.
- **Output contract:** {"payload": "Round Reset"}
- **State tracking:** flow.roundLocked
flow.set("roundLocked", false);
msg.payload = "Round Reset";
node.warn("LOCK RESET = " + flow.get("roundLocked"));
return msg;
- **Algorithm explanation:** The function sets roundLocked to false, allowing another first-player event to be accepted. node.warn() is used to display the resulting lock value during debugging.

Function node: Reset All Scores

- **Purpose:** Reset all player scores.
- **Input contract:** Receives the score-reset event.
- **Output contract:** { "payload": { "P1": 0, "P2": 0, "P3": 0 } }
- **State tracking:** flow.scores
let scores = { P1: 0, P2: 0, P3: 0 };
flow.set("scores", scores);
msg.payload = scores;
return msg;

Algorithm explanation: A new score object is created with all values set to zero. It replaces the previous score state in Flow Context and is sent to the connected score displays.

Function node: Check Answer

- **Purpose:** Validate the answer selected by the player by comparing it with the correct answer stored in the flow context
- **Input contract:** The selected answer received through msg.payload.
- **Output contract— Output 1:** Correct or incorrect answer message.

- **Output contract— Output 2:** null
- **State tracking:** flow.correctAnswer

```
let answer = String(msg.payload).trim();

let correctAnswer = flow.get("correctAnswer");

if (answer === correctAnswer) {

    msg.payload = "✓ الإجابة صحيحة";

} else {

    msg.payload = "✗ الإجابة خاطئة";

}

return [msg, null];
```

Algorithm explanation: The function receives the player's selected answer through msg.payload, converts it to a trimmed string, and retrieves the correct answer from flow.correctAnswer. It then compares both values. If they match, it displays a correct-answer message; otherwise, it displays an incorrect-answer message. The result is sent through Output 1, while Output 2 is not used.

Function node: Questions

- **Purpose:** Store the quiz questions, answer choices, and the correct answer for each question.
- **Input contract:** No input is processed in the provided code.
- **Output contract:** No output is generated in the provided code.
- **State tracking:** No flow state is accessed or modified in the provided code.
-

```
let questions = [

    {
        q: "السؤال الأول ما هو نظام التشغيل مفتوح المصدر؟",
        A: "Windows",
        B: "macOS",
        C: "iOS",
    }
];
```

```
D: "Linux",
correct: "D"
},

{
  q: "السؤال الثاني أي لغة برمجة تستخدم لتطوير تطبيقات Android?",
  A: "Python",
  B: "Swift",
  C: "Java",
  D: "PHP",
  correct: "C"
},

{
  q: "السؤال الثالث ما هي لغة ترميز النصوص التشعبية؟",
  A: "HTML",
  B: "JavaScript",
  C: "CSS",
  D: "PHP",
  correct: "A"
},

{
  q: "السؤال الرابع أي من هذه ليست لغة برمجة؟",
  A: "Python",
  B: "Java",
  C: "HTML",
  D: "C++",
  correct: "C"
},

{
  q: "السؤال الخامس ما هو بروتوكول نقل النص التشعبي؟",
  A: "FTP",
  B: "HTTP",
  C: "SMTP",
  D: "SSH",
  correct: "B"
},
```

```
{
  q: "السؤال السادس أي من هذه قواعد بيانات علائقية؟",
  A: "MongoDB",
  B: "MySQL",
  C: "Redis",
  D: "Firebase",
  correct: "B"
},

{
  q: "localhost المعروف باسم IP السؤال السابع ما هو الـ",
  A: "192.168.1.1",
  B: "10.0.0.1",
  C: "127.0.0.1",
  D: "172.16.0.1",
  correct: "C"
},

{
  q: "السؤال الثامن أي من هذه ليس نظام تشغيل للهواتف؟",
  A: "iOS",
  B: "Android",
  C: "Windows Phone",
  D: "Linux Desktop",
  correct: "D"
},

{
  q: "لـ؟ CSS السؤال التاسع تستخدم لغة الـ",
  A: "تصميم الواجهات",
  B: "البرمجة المنطقية",
  C: "قواعد البيانات",
  D: "الذكاء الاصطناعي",
  correct: "A"
},

{
  q: "السؤال العاشر أي من هذه ليس لغة برمجة نصية؟",
  A: "JavaScript",
  B: "Python",
```

C: "PHP",
D: "C#",
correct: "D"
},

{
q: "السؤال الحادي عشر ما هو بروتوكول البريد الإلكتروني؟",
A: "SMTP",
B: "HTTP",
C: "FTP",
D: "TCP",
correct: "A"
},

{
q: "السؤال الثاني عشر أي من هذه هو إطار عمل جافا سكريبت؟",
A: "React",
B: "Django",
C: "Laravel",
D: "Spring",
correct: "A" }

{
q: "السؤال الثالث عشر ما هي وحدة المعالجة المركزية؟",
A: "الذاكرة",
B: "القرص الصلب",
C: "المعالج",
D: "كارت الشاشة",
correct: "C"
},

{
q: "السؤال الرابع عشر أي من هذه ليس نوع من الذاكرة؟",
A: "RAM",
B: "ROM",
C: "CPU",
D: "Cache",
correct: "C"
},

{

```
q: "DNS؟",
A: "خادم أسماء النطاقات",
B: "بروتوكول نقل",
C: "لغة برمجة",
D: "نظام تشغيل",
correct: "A"
},

{
q: "السؤال السادس عشر أي من هذه يستخدم للتحكم بالإصدارات؟",
A: "Git",
B: "Docker",
C: "Kubernetes",
D: "Jenkins",
correct: "A"
},

{
q: "Apple؟ السؤال السابع عشر ما هو نظام التشغيل من",
A: "Windows",
B: "Linux",
C: "macOS",
D: "Android",
correct: "C"
},

{
q: "السؤال الثامن عشر أي من هذه ليس نوع من قواعد البيانات؟",
A: "علائقية",
B: "وثائقية",
C: "بيانية",
D: "خطية",
correct: "D"
},

{
q: "API؟ السؤال التاسع عشر ما هو الـ",
A: "واجهة برمجة التطبيقات",
B: "لغة برمجة",
```

```

      C: "نظام تشغيل",
      D: "خادم ويب",
      correct: "A"
    },

    {
      q: "السؤال العشرون أي من هذه هو نظام إدارة محتوى؟",
      A: "WordPress",
      B: "React",
      C: "Angular",
      D: "Node.js",
      correct: "A"
    }
  ]
};

```

Algorithm explanation: The function defines an array named questions containing 20 quiz questions. Each question is stored as an object containing the question text (q), four answer choices (A, B, C, and D), and the correct answer (correct). The correct answer varies between questions and is specified individually using the correct property.

Function node: Get Scores

- **Purpose:** Prepare the current game state for the RESTful scores API.
- **Input contract:** HTTP GET request through /api/scores.
- **Output contract:** {"success": true,"scores": {"P1": 0, "P2": 0, "P3": 0 }, "roundLocked": false}
- **State tracking:** flow.scores
flow.roundLocked

```

let scores = flow.get("scores") || {P1: 0,P2: 0, P3: 0};
msg.payload = {
  success: true,
  scores: scores,
  roundLocked: flow.get("roundLocked") || false};
return msg;

```

Algorithm explanation: The function retrieves the stored scores and current lock status and creates a structured JSON object that is returned through the HTTP Response node.

Function node: API Reset Round

- **Purpose:** Reset the round through the RESTful API.
- **Input contract:** HTTP POST request through: /api/reset-round
- **Output contract:** { "success": true, "message": "Round reset successfully", "roundLocked": false }
- **State tracking:** flow.roundLocked

```
flow.set("roundLocked", false);  
msg.payload = {success: true,message: "Round reset  
successfully",roundLocked: false};  
return msg;
```

Algorithm explanation: The function changes the round state to unlocked and creates a structured success response for the HTTP client.

6.1 Communication Protocols

6.1.1 MQTT (if used)

- **Broker URL / IP:** 127.0.0.1
- **Port:** 1883
- **Authentication:** No username/password is configured in the supplied Node-RED broker configuration.
- **TLS / Certificates:** No

Topic	Direction (Subscribe/Publish)	QoS	Retained	Payload Format	Purpose
quiz/winner	Subscribe	0	No	Player number: 1, 2, or 3	Receive the first- player event from ESP32
quiz/reset	Publish	0	No	String: reset	Reset ESP32 for the next round

QoS Justification:

QoS 0 was selected for the current local quiz-game implementation because player events are small and require fast delivery with minimal communication overhead. The trade-off is that QoS 0 does not provide delivery acknowledgement or retransmission if a message is lost.

6.1.2 HTTP (*if used*)

Endpoint	Method	Request Body	Response Body	Status Codes	Purpose
/api/scores	GET	None	success, scores, roundLocked	200 on normal successful response	Retrieve current scores and round state
/api/reset-round	POST	No required body in current flow	success, message, roundLocked	200 on normal successful response	Reset the current round

HTTP Usage:

HTTP is used server-side in Node-RED through HTTP In and HTTP Response nodes. The RESTful endpoints can be tested using Postman and used by the Flutter application to access the required game functions.

6.1.3 WebSockets (*if used*)

Not implemented as a custom project protocol.

No custom WebSocket In or WebSocket Out nodes are present in the supplied Node-RED flow. Therefore, there is no project-specific WebSocket endpoint, message name, or payload to document in this subsection.

6.2 Node Configuration Specs

Node Type	Node Name / Instance	Key Configuration Settings	Purpose / Rationale
MQTT In	ESP32 Winner	Topic: quiz/winner · QoS: 0	Receive the player number published by ESP32
MQTT Out	ESP32 Reset	Topic: quiz/reset · QoS: 0 · Retain: false	Send reset command to ESP32
Switch	Player Selector	msg.payload == P1, P2, or P3 · 3 outputs	Route each player to the appropriate path

Node Type	Node Name / Instance	Key Configuration Settings	Purpose / Rationale
Change	Set MQTT Reset	Set msg.payload to reset	Prepare the MQTT reset message before publishing

6.3 External Services & Integrations

Service	Role in System	Protocol / API	Authentication Method	Data Written / Read
Mosquitto	MQTT message broker between ESP32 and Node-RED	MQTT	None configured	Reads/writes winner and reset messages
MQTTX	MQTT testing and debugging tool	MQTT	According to local broker configuration	Publishes/subscribes to MQTT test messages
Postman	RESTful API testing	HTTP	None configured for current endpoints	Reads scores and tests round reset
Flutter Application	User application layer	RESTful HTTP API	Not specified in supplied information	Reads game information and interacts with game functions

Excluded Services: ThingSpeak, Telegram, WhatsApp, and Email integrations were excluded from the project requirements. Therefore, they are not implemented in the final system.

7. User Interface & Dashboard Design

7.1 Dashboard Overview

- **Purpose:** The dashboard provides a real-time user interface for monitoring and controlling the three-player quiz game. It allows users to view the current question, select an answer, identify the first player, monitor player scores and round status, and reset rounds or scores.

- **Dashboard version / library:** The project uses Node-RED Dashboard 2.0 through the @flowfuse/node-red-dashboard package. The configured package version is 1.30.2, and the dashboard uses the Default Theme.
- **Dashboard structure:** The dashboard contains one main page named Quiz Game, available under the /quiz path. The widgets are organized inside the Quiz Game group. The interface includes the question display, four answer buttons, answer result, first-player display, individual player scores, round status, and reset controls.
- **User workflow:** The user opens the Quiz Game dashboard and views the current question and four answer choices. When one of the three physical players presses a button, the first accepted player is displayed and the round becomes locked. The selected player chooses an answer using A, B, C, or D. The dashboard then displays whether the answer is correct or incorrect. Player scores and the current round status are displayed continuously. The Reset Round and Reset Scores controls are available to prepare a new round or restart the scoring.

7.2 UI Layout & Wireframe — REQUIRED

My Dashboard (Quiz Game)

Quiz Game

السؤال الأول ما هو نظام التشغيل مفتوح المصدر؟

A: Windows B: macOS C: iOS D: Linux

A

B

C

D

START

Player 1 Score

Player 2 Score

Player 3 Score



7.3 UI Node Breakdown — REQUIRED (Table)

Widget	UI Node Type	Purpose	Displayed Data / Unit	Input Topic / Linked Node	User Action → Output Topic
Question Display	ui-text	Display the current quiz question and choices	Question text + A/B/C/D	Check Answer	—
Answer A	ui-button	Submit answer A	A	Check Answer	Click → msg.payload = "A"
Answer B	ui-button	Submit answer B	B	Check Answer	Click → msg.payload = "B"
Answer C	ui-button	Submit answer C	C	Check Answer	Click → msg.payload = "C"
Answer D	ui-button	Submit answer D	D	Check Answer	Click → msg.payload = "D"
Answer Result	ui-text	Display answer validation result	Correct / Incorrect message	First Player Lock	—
First Player Display	ui-text	Display the first accepted player	Player 1 / Player 2 / Player 3	Update Scores / Reset Scores	—

Widget	UI Node Type	Purpose	Displayed Data / Unit	Input Topic / Linked Node	User Action → Output Topic
Player 1 Score	ui-text	Display Player 1 score	payload.P1	Update Scores / Reset Scores	—
Player 2 Score	ui-text	Display Player 2 score	payload.P2	Update Scores / Reset Scores	
Player 3 Score	ui-text	Display Player 3 score	payload.P3	Update Scores / Reset Scores	—
Round Status	ui-text	Display current round state	Ready / Locked	Round Status functions	—
Reset Round	ui-button	Start a new round	reset	Reset Round Lock	Click → reset round and MQTT reset
Reset Scores	ui-button	Clear all player scores	reset_scores	Reset All Scores	Click → reset P1, P2 and P3

8. System Integration & Event Handling This section explains

how the hardware, MQTT broker, Node-RED flow, RESTful APIs, and Dashboard 2.0 work together as one real-time system. It also describes message timing, state management, reconnection behaviour, and the handling of abnormal or unexpected events.

8.1 Synchronization & Timing

- Event-driven handling:** The system follows an event-driven architecture. A physical button press generates an event at the ESP32. The player identifier is published through MQTT and received by Node-RED. The message then passes through player conversion and first-player locking. Dashboard answer-button events are similarly processed when the user selects A, B, C, or D.

The main processing sequence is:

Player Button

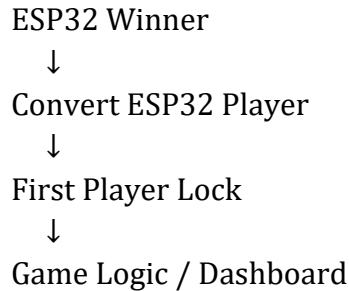
↓

ESP32

↓ MQTT

Mosquitto

↓ MQTT



- **Debounce logic:** Button debounce is implemented in the ESP32 code using a 150 ms debounce interval. This prevents rapid repeated physical button presses from being processed as multiple valid events. Node-RED additionally uses `roundLocked` to prevent a second player from being accepted after the first player has been selected.
- **Rate limiting:** No dedicated Delay node or message rate-limiting configuration is implemented in the current Node-RED flow. The first-player lock and ESP32 debounce logic are used to control repeated player events.
Polling intervals: No periodic polling interval is required for the main ESP32-to-Node-RED communication because MQTT uses event-driven message delivery. Inject nodes are used for manual testing rather than continuous polling.
- **Timestamps / clock synchronisation:** No custom timestamp or clock-synchronisation mechanism is implemented in the current system. The quiz game depends mainly on the order of incoming events and the `roundLocked` state.

8.2 Error Handling & Robustness

The system uses state management, input validation, MQTT reconnection, and debugging mechanisms to improve reliability and prevent invalid game events from affecting the quiz operation. The following subsections describe the implemented robustness mechanisms and the limitations of the current system.

8.2.1 Context Storage

Context Scope (flow / global / environment)	Data Stored	Why This Scope	Lifetime
flow	<code>roundLocked</code>	Required by multiple nodes in the same quiz flow to control first-player access	Maintained during the active Node-RED flow/runtime context

Context Scope (flow / global / environment)	Data Stored	Why This Scope	Lifetime
flow	scores	Allows score- management and API nodes to access the same player scores	Maintained during the active Node-RED flow/runtime context
flow	questionIndex	Tracks the current question	Maintained while the quiz is running
flow	correctAnswer	Stores the configured correct answer	Maintained while processing quiz questions
flow	timeLeft	Stores the configured remaining question time value	Maintained in Flow Context

Flow Context was selected because these values are shared by multiple nodes within the same quiz flow. An external database is not required for the current local prototype.

8.2.2 Catch Node Configuration

- **Scope:** No dedicated Catch node is present in the supplied Node-RED flow.
- **Action on error:** Debug nodes and Function-node logging are used during development to inspect messages and diagnose incorrect system behaviour.
- **Error output used for:** No dedicated Catch-node fallback or automatic retry output is implemented in the current version.

8.2.3 Reconnect Logic

- **MQTT reconnect:** Automatic. The MQTT broker configuration uses Auto Connect, allowing Node-RED to reconnect to the MQTT broker when the connection becomes available.
- **HTTP retry:** No explicit HTTP retry or backoff mechanism is implemented in the current Node-RED flow.
- **Status monitoring:** Node-RED node-status indicators and Debug nodes are used during development to check connection status and message flow.

8.2.4 Edge-Case Handling

Edge Case	How It Is Detected	Handling Strategy
Missing / malformed player payload	Convert ESP32 Player checks whether the payload can represent Player 1, 2, or 3	Invalid input is discarded using return null
Invalid player identifier	First Player Lock checks for P1, P2, or P3	Invalid identifiers are discarded using return null

Edge Case	How It Is Detected	Handling Strategy
Duplicate player press	roundLocked === true	Ignore additional player messages until the round is reset
Scores not initialized	flow.get("scores") returns no stored score object	Initialize scores as {P1:0, P2:0, P3:0}
End of question list	questionIndex reaches the number of available questions	Display the end-of-questions message
Broker offline	MQTT connection becomes unavailable	MQTT Auto Connect attempts reconnection when the broker becomes available

9. Testing, Validation & Grading Evidence

This section presents the testing and validation of the IoT quiz game. The system components are tested individually and as an integrated system using Node-RED Debug, MQTTX, Postman, Dashboard 2.0, and the ESP32 hardware. Test cases and screenshots are used as evidence to verify the implemented system functions.

9.1 Testing Strategy

The system was tested at three levels:

1. **Unit testing — individual nodes / isolated flow segments:** Individual Node-RED nodes and functions were tested using Inject and Debug nodes. Test payloads such as P1, P2, and P3 were injected to verify player selection, First Player Lock, score updates, round reset, score reset, question display, and answer checking.
2. **Integration testing — between layers:** Communication between the ESP32, Mosquitto MQTT broker, Node-RED, and Dashboard 2.0 was tested. MQTTX was used to verify MQTT publish/subscribe messages, while Postman was used to test the RESTful API endpoints.
3. **End-to-end scenario testing:** The complete quiz-game sequence was tested from a physical player button press through the ESP32 and MQTT to Node-RED. The tests verified first-player detection, round locking, answer processing, dashboard updates, score handling, and round reset.

Test environment:

A hybrid testing environment was used. Real ESP32 hardware, physical push buttons, LEDs, and a buzzer were combined with Node-RED software testing using Inject and Debug nodes. MQTTX and Postman were used for protocol and API verification.

9.2 Debugging Evidence — REQUIRED (Screenshot)

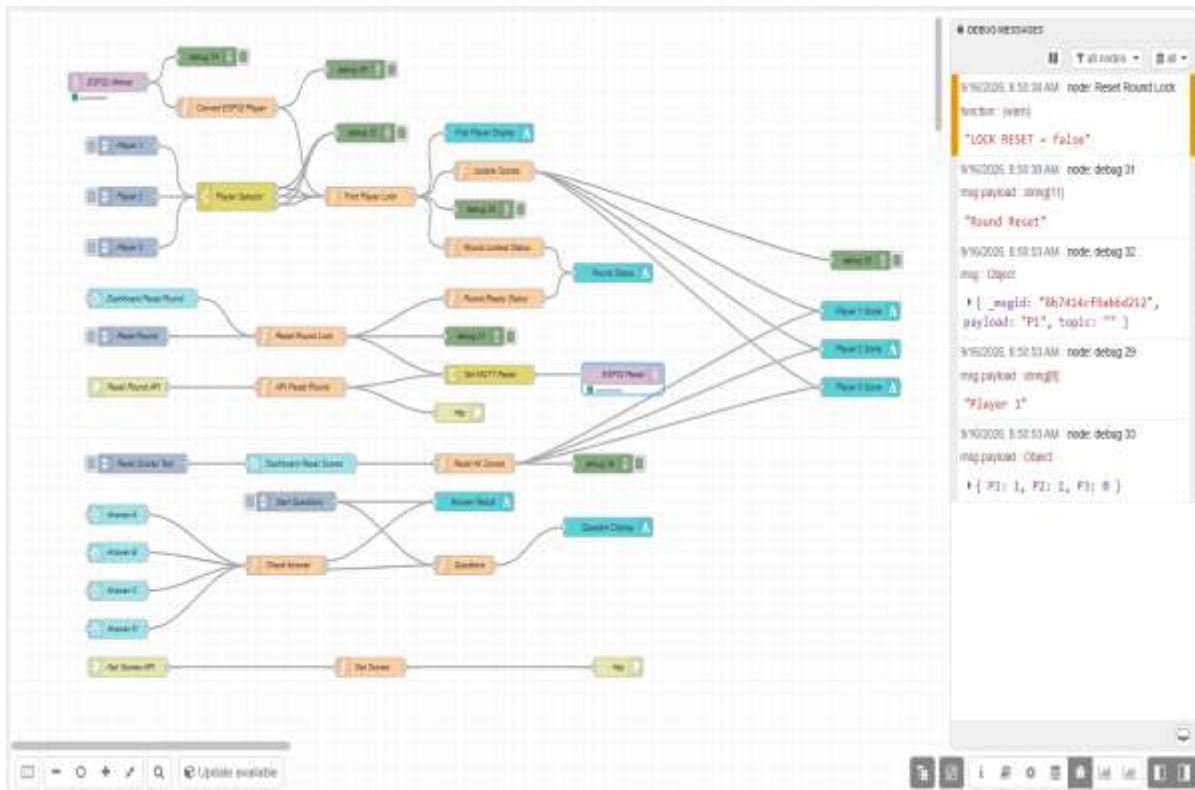


Figure 4. Node-RED Debug Evidence for Player Detection and Round Processing.

Stage: First-player detection and game-state processing.

Payload Evidence: Player 1, Round Reset, and {P1: 1, P2: 1, P3: 0}.

Proof: The Debug panel confirms that the round reset was processed, Player 1 was successfully detected as the first player, and the current player scores were generated by the Node-RED flow.

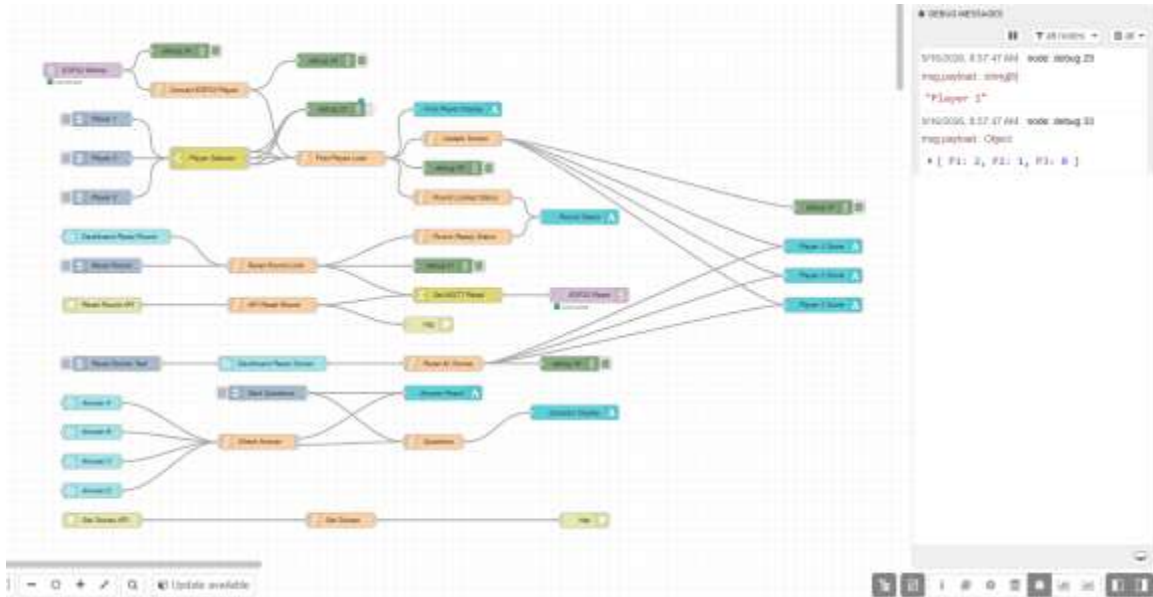


Figure 5. First Player Lock Validation in Node-RED Debug Panel.

Stage: First-player locking mechanism.

Test: Player 1, Player 2, and Player 3 were triggered sequentially during the same round.

Output: Only Player 1 passed through the First Player Lock.

Proof: The Debug panel confirms that the first player was accepted while subsequent player events were blocked during the same round.

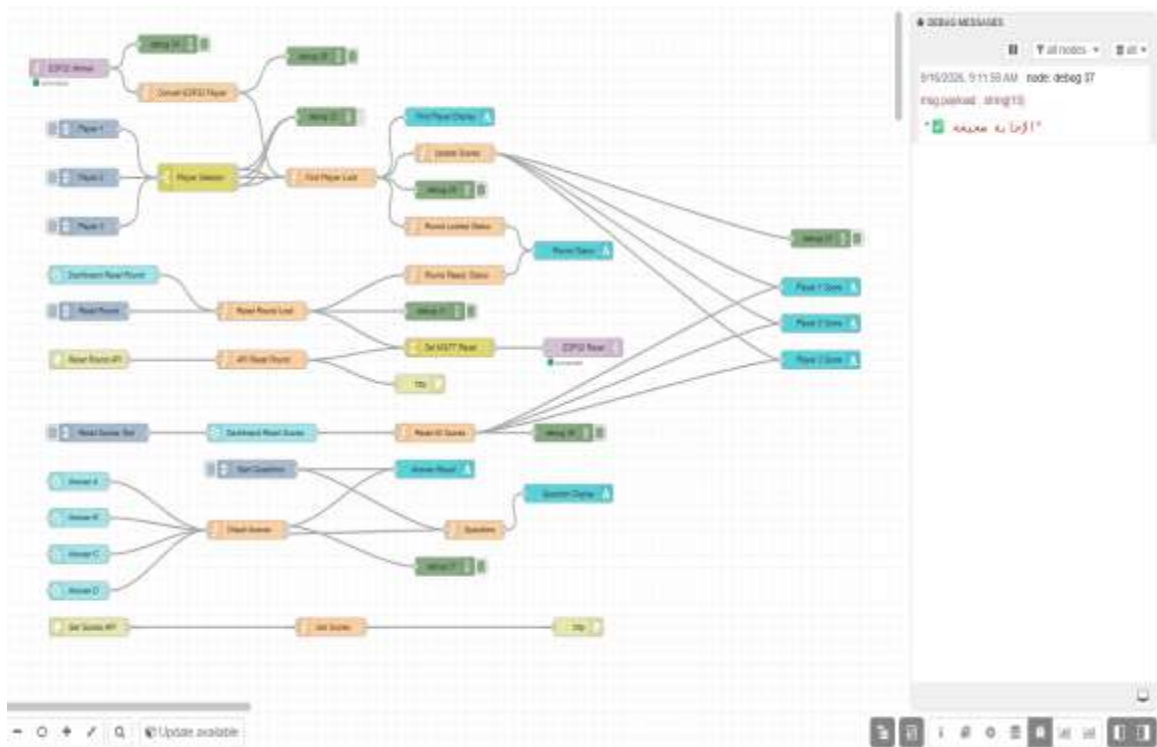


Figure 6. Correct Answer Validation in Node-RED Debug Panel.

Stage: **Answer validation.**

Input: **correct answer.**

Output: ✓ الإجابة صحيحة (Correct Answer).

Proof: For the tested question, Answer C was configured as the correct answer, and the Debug panel confirms that the Check Answer function validated it correctly.

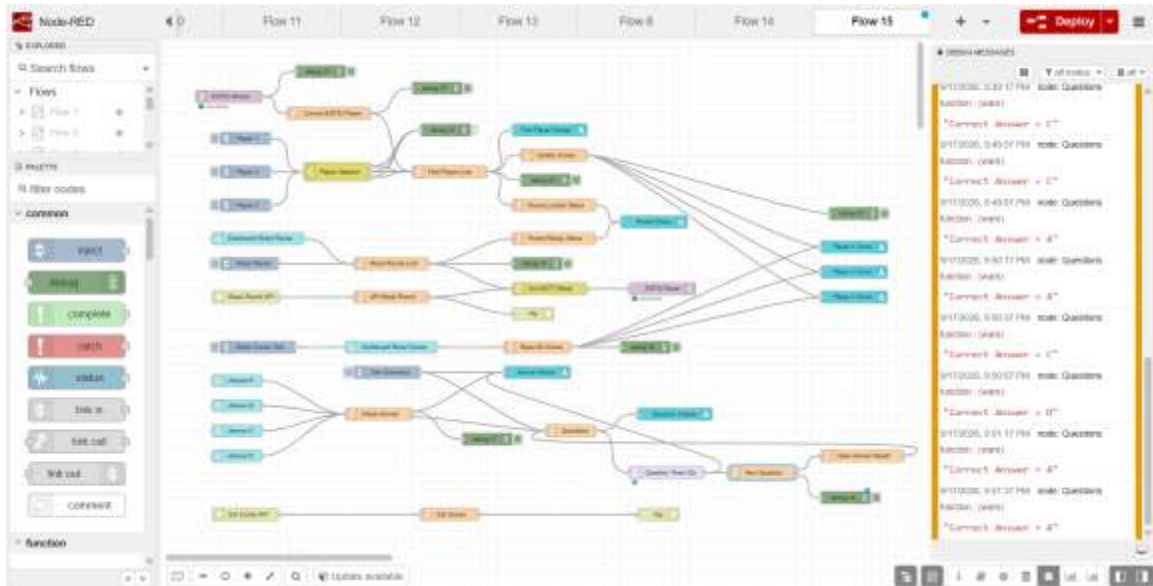


Figure 8. Automatic Question Transition After 20 Seconds.

Stage: Question timer and automatic transition.

Input: The current question is displayed for 20 seconds.

Output: The next question is loaded automatically.

Proof: The Debug panel shows consecutive question updates at 20-second intervals, confirming that the timer and automatic question transition operate correct

Quiz Game

السؤال الأول ما هو نظام التشغيل مفتوح المصدر؟

A: Windows B: macOS C: iOS D: Linux

A

B

C

D

الإجابة صحيحة ✓

Player 1 Score 0

Player 2 Score 0

Player 3 Score 0



Figure 9. Answer Result Clearing After Automatic Question Transition.

Stage: Answer result clearing and question transition.

Before Transition: The correct-answer message is displayed for the current question.

After Transition: After 20 seconds, the next question is displayed and the previous answer result is cleared automatically.

Proof: For the tested question, Answer C was configured as the correct answer, and the Debug panel confirms that the Check Answer function validated it correctly.

9.3 Test Cases — REQUIRED (Table)

Test Case ID	Input Event / Msg Payload	Expected Output / UI State	Actual Result	Pass/Fail
TC-01	ESP32 sends player 1	Node-RED converts 1 to P1 and identifies Player 1	Player 1 was detected by Node-RED	Pass

Test Case ID	Input Event / Msg Payload	Expected Output / UI State	Actual Result	Pass/Fail
TC-02	P1 presses first, followed by P2/P3	P1 is accepted and later presses are ignored	First Player Lock accepted only the first player	Pass
TC-03	Reset Round command	roundLocked changes to false and a new player can press	Round was unlocked and prepared for the next player	Pass
TC-04	Answer button payload = C	Dashboard displays ✓ الإجابة صحيحة and advances to next question	Correct-answer message was displayed	Pass
TC-05	Answer payload = A, B, or D	Dashboard displays ✗ الإجابة خاطئة	Incorrect-answer message was displayed	Pass
TC-06	Reset Scores command	P1, P2, and P3 scores become 0	All displayed scores were reset to zero	Pass
TC-07	GET /api/scores using Postman	JSON response contains success, scores, and roundLocked	API returned the current game state	Pass
TC-08	POST /api/reset-round using Postman	API returns success and roundLocked:false	Round-reset response was returned	Pass

9.4 Validation Against Objectives

Objective (from 3.3)	Proving Test Case(s)	Evidence (screenshot/table)
Develop an IoT quiz game for three players	TC-01, TC-02	Figures 4–5 and Table 9.3
Connect the ESP32 with Node-RED using MQTT	TC-01	Figure 4 and MQTT Debug output
Detect and lock the first player who presses a button	TC-02, TC-03	Figure 5 and Debug output

Objective (from 3.3)	Proving Test Case(s)	Evidence (screenshot/table)
Manage questions, answers, and player scores	TC-04, TC-05, TC-06	Dashboard and Debug screenshots
Display the game status using Dashboard 2.0	TC-02, TC-04, TC-06	Dashboard screenshot
Provide RESTful APIs for Flutter integration	TC-07, TC-08	Postman screenshots
Test the system using MQTTX, Postman, and Node-RED Debug	TC-01–TC-08	Figures 4–7 and API/MQTT test evidence

10. Results and Performance Discussion

10.1 Functional Evaluation

The implemented system was tested using the ESP32 hardware, Mosquitto MQTT broker, Node-RED, Dashboard 2.0, MQTTeX, and Postman. The main communication and processing functions operated successfully during the performed tests.

The ESP32 was able to generate player events, and Node-RED received and processed the player information through MQTT. The Convert ESP32 Player function converted the received player number into the required internal format, while the First Player Lock function used `roundLocked` to accept the first player and reject additional player events during the same round.

The question and answer section displayed the quiz questions and allowed answers to be submitted using the A, B, C, and D Dashboard buttons. The answer-checking logic compares the selected answer with the correct answer stored in `flow.correctAnswer`. If the selected answer matches the correct answer, the system displays the correct-answer result; otherwise, it displays the incorrect-answer result.

The Dashboard displayed the current question, answer result, first player, round status, and player scores. Reset functions were also tested to unlock the round and reset player scores.

MQTT communication was tested using MQTTeX, and the RESTful API endpoints were tested using Postman. The `GET /api/scores` endpoint was used to retrieve the current scores and round-lock state, while `POST /api/reset-round` was used to reset the current round.

A deviation exists between the intended scoring behaviour and the current Node-RED flow arrangement. The intended game behaviour is to award a point only after a correct answer. However, the current Update Scores logic increments the selected player's score when it receives the player-selection message. Therefore, the score-update path should be connected to successful answer validation in the final implementation if points must only be awarded for correct answers.

The functional results should be evaluated together with the test cases presented in Section 9.3.

10.2 Latency & Performance

Measurement method:

No formal numerical latency measurement using timestamp differences, browser-console measurements, or dedicated performance-monitoring nodes was recorded in the supplied project information. Therefore, numerical latency and throughput values cannot be reported reliably.

- **End-to-end message latency:** Not formally measured. During the performed local functional tests, player events were observed to propagate from the input and MQTT communication layer to Node-RED and the Dashboard interactively, but no measured average latency in milliseconds is available.
- **UI update responsiveness:** Dashboard 2.0 was used to display player information, questions, answer results, scores, and round status. The supplied project information confirms functional Dashboard testing, but does not contain a numerical UI response-time measurement.
- **Broker / flow throughput:** Not formally measured. No documented messages-per-second throughput test was performed for the Mosquitto broker or Node-RED flow.
- **Test environment:** The system was tested using the physical ESP32 quiz-game hardware together with Node-RED, a local Mosquitto MQTT broker, Dashboard 2.0, MQTTeX, Postman, and the Arduino development environment. The supplied project information does not include the host computer specifications required for a detailed performance benchmark.

10.3 System Limitations

Memory / CPU bottlenecks: No formal CPU or memory stress test was performed. The current system stores game state such as scores, roundLocked, and questionIndex in Node-RED Flow Context. Therefore, long-duration CPU and memory performance cannot be quantified from the available test evidence.

API / service rate limits: The current implementation does not use a documented third-party cloud API with a defined request-rate limit. The implemented RESTful endpoints are handled locally by Node-RED.

Flow complexity / maintainability: The main project functions are contained within one Node-RED flow named Flow 15. This is suitable for the current prototype, but increasing the number of questions, players, communication interfaces, or game functions could make the flow more difficult to maintain and debug. Future versions could divide the system into separate tabs or reusable subflows.

Single points of failure: The current architecture depends on a single Node-RED runtime and a single local Mosquitto MQTT broker. If either component becomes unavailable, MQTT communication and game processing may be interrupted.

Other: The current implementation has several additional limitations:

- The answer-checking logic depends on the correct answer stored in `flow.correctAnswer`. Therefore, the correct answer must be properly set for each question before the player's selected answer is validated.
- The intended scoring rule is to award a point after a correct answer, while the current flow arrangement updates the score from the player-selection path. This should be corrected in the final flow if strict correct-answer scoring is required.
- No persistent external database is implemented for long-term score storage.
- MQTT TLS is not enabled in the current local broker configuration.
- No authentication mechanism is documented for the current RESTful API endpoints.
- No formal numerical latency, throughput, CPU, or memory performance measurements were recorded.

11. Challenges, Debugging & Trade-offs

11.1 Technical Roadblocks

During the development and integration of the IoT quiz game, several technical problems were encountered. The most significant problems, their identified causes, and the solutions applied are summarized below.

#	Symptom	Root Cause	Resolution
1	Physical player button presses were not initially received correctly in the Node-RED Dashboard	The MQTT communication path between the ESP32, broker, and Node-RED required verification	The MQTT publisher/subscriber topics were checked using MQTTX and Node-RED Debug, and the player payload was processed using the Convert ESP32 Player Function
2	More than one player could be processed during	The system required a stored state to determine whether a	The roundLocked Flow Context variable and First Player Lock

#	Symptom	Root Cause	Resolution
	the same quiz round	player had already pressed first	Function were implemented to accept only the first player
3	Some selected answers initially produced an incorrect validation result.	The answer-validation logic was not correctly synchronized with the correct answer of the current question.	The Check Answer logic was modified to compare the selected answer with <code>flow.correctAnswer</code> .

Roadblock 1: ESP32-to-Node-RED Communication

During hardware and software integration, physical button events from the ESP32 did not initially appear correctly in the Node-RED Dashboard. The communication path was divided into separate stages for testing.

The Arduino Serial Monitor was first used to verify that the ESP32 detected physical button presses. MQTTS was then used to test MQTT publishing and subscription independently. Finally, Node-RED Debug nodes were used to inspect the received payload.

This process helped verify that the MQTT publisher and subscriber were using matching topics and that Node-RED was receiving the expected player value. The **Convert ESP32 Player** Function was then used to convert the numeric ESP32 values into the internal player format.

Roadblock 2: Multiple Player Events

The game requires only the first player who presses a button to be accepted during each round. Without a stored lock state, later player events could continue to be processed.

The problem was resolved using the `roundLocked` Flow Context variable. The First Player Lock Function checks the variable before accepting a player:

```
let locked = flow.get("roundLocked") || false;

if (locked === true) {
  return null;
}

flow.set("roundLocked", true);
```

When the first valid player is accepted, `roundLocked` becomes `true`. Any later player message is discarded using `return null`. The value is returned to `false` when the round is reset.

Roadblock 3: Incorrect Answer Validation

During Dashboard testing, some selected answers initially produced an incorrect validation result because the answer-checking logic was not correctly synchronized with the correct answer of the current question.

The answer-validation logic was corrected by retrieving the correct answer from Flow Context and comparing it with the selected answer:

```
let correctAnswer = flow.get("correctAnswer");
```

```
if (answer === correctAnswer)
```

After this modification, the system validates each selected answer according to the correct answer configured for the current question.

11.2 Debugging Techniques Used

The following debugging techniques were used during development:

- **Inject node with mock payloads** — Inject nodes were used to simulate player events such as P1, P2, and P3. This allowed sections of the Node-RED flow to be tested without repeatedly using the physical hardware.
- **Debug node inspection** — Debug nodes were connected to important stages of the flow to inspect `msg.payload` and, when required, the complete message object. This helped verify player values, MQTT messages, score objects, and reset operations.
- **Node status colours** — Node status indicators were observed during MQTT testing to identify whether communication nodes were connected or experiencing a connection problem.
- **Context inspection** — Flow Context values were checked during testing, particularly:

```
roundLocked  
scores  
questionIndex
```

This helped verify that the internal game state changed correctly between rounds and questions.

- **External tools** — MQTTX was used to independently test MQTT publishing and subscription. This helped determine whether communication problems originated from the ESP32, MQTT broker, or Node-RED.

Postman was used to test the RESTful API endpoints independently:

GET /api/scores
POST /api/reset-round

This helped verify the HTTP layer without depending on the Flutter application.

- **Arduino Serial Monitor** — The Serial Monitor was used to verify physical button events and ESP32 operation before checking whether the same events reached MQTT and Node-RED.

11.3 Design Trade-offs

Decision Taken	Alternative Considered	Reason / Trade-off
Store game state in Node-RED Flow Context	External database	Flow Context is simple and suitable for the current prototype. However, it does not provide the same long-term persistence and scalability as an external database
Use MQTT QoS 0	MQTT QoS 1 or QoS 2	QoS 0 reduces communication overhead and is suitable for fast local interaction. The trade-off is that message delivery is not acknowledged or guaranteed
Use MQTT push communication for player events	HTTP polling	MQTT provides event-driven communication and avoids unnecessary repeated HTTP requests
Use Dashboard 2.0	Dashboard 1.0	Dashboard 2.0 satisfies the project requirement and provides the required Node-RED user interface
Use a local Mosquitto MQTT broker	Cloud-hosted MQTT broker	A local broker simplifies configuration and testing. The trade-off is reduced remote accessibility and dependence on the local environment
Store the project logic mainly in one Node-RED flow	Multiple tabs or reusable subflows	A single flow simplifies the current prototype and demonstration. However, modular flows would be easier to maintain as the project becomes larger

11.4 Lessons Learned

- **Test system components individually before complete integration.** Testing the ESP32, MQTT broker, Node-RED, Dashboard, and REST API separately made it easier to locate communication and logic problems.
- **Define message formats and MQTT topics consistently.** Maintaining consistent player representations such as 1 → P1 → Player 1 and matching MQTT topics between publisher and subscriber reduced integration errors.
- **Use state management in event-driven applications.** The roundLocked variable demonstrated how Flow Context can be used to control system behaviour when several players may generate events within a short period.
- **Use multiple debugging tools to isolate problems.** Node-RED Debug, Inject nodes, MQTTX, Postman, and the Arduino Serial Monitor were useful for testing different layers of the system independently.
- **Modularize the system as complexity increases.** The current single-flow structure is manageable for the prototype, but separating MQTT communication, game logic, REST APIs, and Dashboard processing into multiple flows or subflows would improve maintainability and debugging in a larger version.

12. Conclusion & Future Improvements

12.1 Summary of Achievements

The project successfully developed an IoT-based three-player quiz game integrating physical hardware with Node-RED, MQTT, Dashboard 2.0, RESTful APIs, and a Flutter application.

The achievement of the project objectives is summarized as follows:

- Develop an IoT quiz game for three players — Fully Met.

The system supports three players using physical push buttons connected to the ESP32 and processes player events through Node-RED.

- Connect the ESP32 with Node-RED using MQTT — Fully Met.

MQTT communication was implemented through a Mosquitto broker using the quiz/winner and quiz/reset topics.

- Detect and lock the first player who presses a button — Fully Met.

The roundLocked Flow Context variable and the First Player Lock Function allow the system to accept the first player and ignore additional player events during the same round.

- Manage questions, answers, and player scores — Partially Met.

The system successfully manages quiz questions, stores the correct answer for each question in flow.correctAnswer, validates the player's selected answer, and maintains player scores using Node-RED Flow Context. However, in the current flow, the Update Scores function is connected to the first-player selection path, causing the selected player's score to increase when the first player is registered rather than only after a correct answer. Therefore, the scoring path requires further adjustment to fully match the intended game rule.

- Display the game status using Dashboard 2.0 — Fully Met.

The Dashboard provides question display, answer buttons, answer results, first-player information, player scores, round status, and reset controls.

- Provide RESTful APIs for Flutter integration — Fully Met.

Node-RED provides GET /api/scores for retrieving game state and POST /api/reset-round for resetting the current round.

- Test the system using MQTTX, Postman, and Node-RED Debug — Fully Met.

These tools were used during development to test MQTT communication, RESTful APIs, message processing, and game-state behaviour.

Overall, the project achieved the main objective of integrating physical player interaction with an event-driven IoT software architecture. The main functional components operate together successfully. Question-specific correct answers are managed dynamically through flow.correctAnswer, while the score-update sequence remains an identified area for future improvement.

12.2 Scalability

The current system is designed as a three-player local prototype. Several improvements could be introduced to support a larger system:

- **Onboarding many devices:** The MQTT topic structure could be extended to support additional ESP32 devices, players, or multiple quiz-game units.
- **Modularising flows into reusable subflows:** MQTT communication, player management, question processing, scoring, and REST API functions could be separated into reusable Node-RED subflows. This would reduce flow complexity and simplify maintenance.
- **Offloading state to a database:** Player scores, questions, game results, and historical data could be stored in an external database instead of relying only on Node-RED Flow Context.
- **Load-splitting across multiple Node-RED instances:** For a larger deployment, processing could be distributed across multiple Node-RED instances rather than depending on one local runtime.

12.3 Cloud Deployment Strategies

The current implementation operates mainly in a local environment. Future versions could use cloud deployment to improve remote accessibility and scalability.

Possible deployment strategies include:

- **Docker / container deployment:** Node-RED and related services could be deployed using containers to provide a consistent and portable runtime environment.
- **Managed or cloud-hosted Node-RED:** The Node-RED runtime could be hosted remotely so that the quiz system can be accessed outside the local network.
- **Cloud-hosted MQTT broker:** The local Mosquitto broker could be replaced or supplemented by a cloud-hosted MQTT broker to support remote ESP32 devices.
- **Persistent external database:** A cloud or server-based database could store player scores, questions, game results, and historical records.
- **Monitoring and alerting:** Future deployments could include service-health monitoring and alerts for broker, Node-RED, or device connection failures.

- **CI/CD for flow deployment:** Node-RED flow files could be maintained using version control and an automated deployment process to improve change management.

12.4 Security Enhancements

The current prototype focuses primarily on functionality. Future versions should introduce additional security mechanisms, including:

- **TLS for MQTT and HTTP:** MQTT communication could use TLS instead of plain TCP on port 1883, and RESTful API communication could use HTTPS.
- **Authentication for dashboards and APIs:** Access to Dashboard 2.0 and RESTful API endpoints could be protected using authentication or token-based access.
- **Encrypted credential management:** MQTT credentials, API tokens, and other sensitive information should be stored using environment variables or the Node-RED credential store instead of being written directly inside flows or source code.
- **MQTT topic Access Control Lists (ACLs):** Permissions could restrict which devices are allowed to publish or subscribe to topics such as quiz/winner and quiz/reset.
- **HTTP rate limiting:** Rate limits could be applied to RESTful API endpoints to reduce excessive or unauthorized requests.
- **Secure remote dashboard access:** If the Dashboard is deployed outside the local network, secure authenticated remote access should be implemented.

13. References

[1] Node-RED, "Node-RED Documentation." Available:
<https://nodered.org/docs/>

[2] FlowFuse, "Node-RED Dashboard 2.0 Documentation." Available:
<https://dashboard.flowfuse.com/>

[3] OASIS, "MQTT Version 3.1.1," OASIS Standard. Available:
<https://docs.oasis-open.org/mqtt/mqtt/v3.1.1/mqtt-v3.1.1.html>

[4] Eclipse Foundation, "Eclipse Mosquitto Documentation." Available:
<https://mosquitto.org/documentation/>

[5] Arduino, "Arduino Documentation." Available:
<https://docs.arduino.cc/>

[6] Postman, "Postman Learning Center." Available:
<https://learning.postman.com/docs/>

14. Appendices

Appendix A: Complete Node-RED Flow Export

The complete Node-RED flow used to implement the IoT-based three-player quiz game is included in this appendix. The flow contains MQTT communication, player conversion, first-player locking, score management, question and answer processing, round reset, RESTful APIs, Dashboard 2.0 nodes, and debugging nodes.

Node-RED Flow Name: Flow 15

Complete Node-RED Flow JSON:



flows (6).json

```
[
  {
    "id": "72b54a41227aca2a",
    "type": "tab",
    "label": "Flow 15",
    "disabled": false,
    "info": "",
    "env": []
  },
  {
    "id": "71880a0dae56e60c",
    "type": "inject",
    "z": "72b54a41227aca2a",
    "name": "Player 1",
    "props": [
      {
        "p": "payload"
      }
    ]
  }
]
```

```

        },
        {
            "p": "topic",
            "vt": "str"
        }
    ],
    "repeat": "",
    "crontab": "",
    "once": false,
    "onceDelay": 0.1,
    "topic": "",
    "payload": "P1",
    "payloadType": "str",
    "x": 240,
    "y": 260,
    "wires": [
        [
            "1abac244f86c819d"
        ]
    ]
},
{
    "id": "e8c0a7a28700f12b",
    "type": "inject",
    "z": "72b54a41227aca2a",
    "name": "Player 2",
    "props": [
        {
            "p": "payload"
        },
        {
            "p": "topic",
            "vt": "str"
        }
    ],
    "repeat": "",
    "crontab": "",

```

```
    "once": false,
    "onceDelay": 0.1,
    "topic": "",
    "payload": "P2",
    "payloadType": "str",
    "x": 240,
    "y": 340,
    "wires": [
      [
        "1abac244f86c819d"
      ]
    ]
  },
  {
    "id": "6bc7271e3555ce6f",
    "type": "inject",
    "z": "72b54a41227aca2a",
    "name": "Player 3",
    "props": [
      {
        "p": "payload"
      },
      {
        "p": "topic",
        "vt": "str"
      }
    ]
  },
  "repeat": "",
  "crontab": "",
  "once": false,
  "onceDelay": 0.1,
  "topic": "",
  "payload": "P3",
  "payloadType": "str",
  "x": 240,
  "y": 420,
  "wires": [
```

```

        [
            "1abac244f86c819d"
        ]
    ],
    {
        "id": "1abac244f86c819d",
        "type": "switch",
        "z": "72b54a41227aca2a",
        "name": "Player Selector",
        "property": "payload",
        "propertyType": "msg",
        "rules": [
            {
                "t": "eq",
                "v": "P1",
                "vt": "str"
            },
            {
                "t": "eq",
                "v": "P2",
                "vt": "str"
            },
            {
                "t": "eq",
                "v": "P3",
                "vt": "str"
            }
        ],
        "checkall": "true",
        "repair": false,
        "outputs": 3,
        "x": 460,
        "y": 340,
        "wires": [
            "696d4cd71972b047",

```

```

        "5e4fdb988cdc3300"
    ],
    [
        "696d4cd71972b047",
        "5e4fdb988cdc3300"
    ],
    [
        "696d4cd71972b047",
        "5e4fdb988cdc3300"
    ]
]
},
{
    "id": "696d4cd71972b047",
    "type": "function",
    "z": "72b54a41227aca2a",
    "name": "First Player Lock",
    "func": "let locked = flow.get(\"roundLocked\") ||
false;\n\n// إذا لاعب سبق وضغط، تجاهل أي لاعب آخر\n\nif (locked ===
true) {\n    return null;\n}\n\n// أول لاعب فقط يتم
تسجيله\n\nflow.set(\"roundLocked\", true);\n\nif (msg.payload
=== \"P1\") {\n    msg.payload = \"Player 1\";\n}\n\nelse if
(msg.payload === \"P2\") {\n    msg.payload = \"Player
2\";\n}\n\nelse if (msg.payload === \"P3\") {\n    msg.payload
= \"Player 3\";\n}\n\nelse {\n    return null;\n}\n\nreturn
msg;",
    "outputs": 1,
    "timeout": 0,
    "noerr": 0,
    "initialize": "",
    "finalize": "",
    "libs": [],
    "x": 730,
    "y": 340,
    "wires": [
        "527dd499c13757c0",

```

```

        "0f4f17d66b1e3b53",
        "9c2bbb531a6be0c6",
        "9beb1883642ad027"
    ]
]
},
{
    "id": "6f270be948f2897e",
    "type": "inject",
    "z": "72b54a41227aca2a",
    "name": "Reset Round",
    "props": [
        {
            "p": "payload"
        },
        {
            "p": "topic",
            "vt": "str"
        }
    ],
    "repeat": "",
    "crontab": "",
    "once": false,
    "onceDelay": 0.1,
    "topic": "",
    "payload": "RESET",
    "payloadType": "str",
    "x": 250,
    "y": 560,
    "wires": [
        [
            "eda118db274e4d79"
        ]
    ]
},
{
    "id": "eda118db274e4d79",

```

```

        "type": "function",
        "z": "72b54a41227aca2a",
        "name": "Reset Round Lock",
        "func": "flow.set(\"roundLocked\",
false);\n\nmsg.payload = \"Round Reset\";\n\nnode.warn(\"LOCK
RESET = \" +
flow.get(\"roundLocked\")); \nglobal.get\ncontext.get\n\nretu
rn msg;",
        "outputs": 1,
        "timeout": 0,
        "noerr": 0,
        "initialize": "",
        "finalize": "",
        "libs": [],
        "x": 590,
        "y": 560,
        "wires": [
            [
                "e052088121fe5f35",
                "97e14cca3ad9a8e0",
                "7c7157e86b859c86"
            ]
        ]
    },
    {
        "id": "527dd499c13757c0",
        "type": "debug",
        "z": "72b54a41227aca2a",
        "name": "debug 29",
        "active": true,
        "tosidebar": true,
        "console": false,
        "tostatus": false,
        "complete": "payload",
        "targetType": "msg",
        "statusVal": "",
        "statusType": "auto",

```

```
    "x": 960,  
    "y": 360,  
    "wires": []  
  },  
  {  
    "id": "7c7157e86b859c86",  
    "type": "debug",  
    "z": "72b54a41227aca2a",  
    "name": "debug 31",  
    "active": true,  
    "tosidebar": true,  
    "console": false,  
    "tostatus": false,  
    "complete": "false",  
    "statusVal": "",  
    "statusType": "auto",  
    "x": 940,  
    "y": 560,  
    "wires": []  
  },  
  {  
    "id": "5e4fdb988cdc3300",  
    "type": "debug",  
    "z": "72b54a41227aca2a",  
    "name": "debug 32",  
    "active": false,  
    "tosidebar": true,  
    "console": false,  
    "tostatus": false,  
    "complete": "true",  
    "targetType": "full",  
    "statusVal": "",  
    "statusType": "auto",  
    "x": 720,  
    "y": 240,  
    "wires": []  
  },  
}
```

```

{
  "id": "de9fa5db48f6e96b",
  "type": "mqtt in",
  "z": "72b54a41227aca2a",
  "name": "ESP32 Winner",
  "topic": "quiz/winner",
  "qos": "0",
  "datatype": "auto-detect",
  "broker": "c4575839817bb6d1",
  "nl": false,
  "rap": true,
  "rh": 0,
  "inputs": 0,
  "x": 200,
  "y": 160,
  "wires": [
    [
      "35f8871f40d1d748",
      "61e7867d2c2949f5"
    ]
  ]
},
{
  "id": "35f8871f40d1d748",
  "type": "function",
  "z": "72b54a41227aca2a",
  "name": "Convert ESP32 Player",
  "func": "let p = Number(msg.payload);\n\nif (p ===
1) {\n  msg.payload = \"P1\";\n} else if (p === 2)
{\n  msg.payload = \"P2\";\n} else if (p === 3)
{\n  msg.payload = \"P3\";\n} else {\n  return
null;\n}\n\nreturn msg;",
  "outputs": 1,
  "timeout": 0,
  "noerr": 0,
  "initialize": "",
  "finalize": "",

```

```
    "libs": [],
    "x": 440,
    "y": 200,
    "wires": [
      [
        "696d4cd71972b047",
        "e9d05f69d9f5670c"
      ]
    ]
  },
  {
    "id": "2a21c38ba4936815",
    "type": "mqtt out",
    "z": "72b54a41227aca2a",
    "name": "ESP32 Reset",
    "topic": "quiz/reset",
    "qos": "0",
    "retain": "false",
    "respTopic": "",
    "contentType": "",
    "userProps": "",
    "correl": "",
    "expiry": "",
    "broker": "c4575839817bb6d1",
    "x": 1240,
    "y": 620,
    "wires": []
  },
  {
    "id": "e052088121fe5f35",
    "type": "change",
    "z": "72b54a41227aca2a",
    "name": "Set MQTT Reset",
    "rules": [
      {
        "t": "set",
        "p": "payload",
```

```

        "pt": "msg",
        "to": "reset",
        "tot": "str"
    }
],
"action": "",
"property": "",
"from": "",
"to": "",
"reg": false,
"x": 970,
"y": 620,
"wires": [
    [
        "2a21c38ba4936815"
    ]
]
},
{
    "id": "0f4f17d66b1e3b53",
    "type": "function",
    "z": "72b54a41227aca2a",
    "name": "Update Scores",
    "func": "let scores = flow.get(\"scores\") ||
{\n    P1: 0,\n    P2: 0,\n    P3: 0\n};\n\nif (msg.payload
=== \"Player 1\") {\n    scores.P1++;\n}\nelse if
(msg.payload === \"Player 2\") {\n    scores.P2++;\n}\nelse
if (msg.payload === \"Player 3\")
{\n    scores.P3++;\n}\n\nflow.set(\"scores\",
scores);\n\nmsg.payload = scores;\n\nreturn msg;",
    "outputs": 1,
    "timeout": 0,
    "noerr": 0,
    "initialize": "",
    "finalize": "",
    "libs": [],
    "x": 980,

```

```
    "y": 300,  
    "wires": [  
      [  
        "190b2c798d01e8fe",  
        "96346da906edd097",  
        "6569064ea7fa08d9",  
        "db21b37b840bbb57"  
      ]  
    ]  
  },  
  {  
    "id": "96346da906edd097",  
    "type": "debug",  
    "z": "72b54a41227aca2a",  
    "name": "debug 33",  
    "active": true,  
    "tosidebar": true,  
    "console": false,  
    "tostatus": false,  
    "complete": "false",  
    "statusVal": "",  
    "statusType": "auto",  
    "x": 1680,  
    "y": 460,  
    "wires": []  
  },  
  {  
    "id": "190b2c798d01e8fe",  
    "type": "ui-text",  
    "z": "72b54a41227aca2a",  
    "group": "0851430a4244eecb",  
    "order": 8,  
    "width": 0,  
    "height": 0,  
    "name": "Player 1 Score",  
    "label": "Player 1 Score",  
    "format": "{{msg.payload}}",
```

```
    "layout": "row-spread",
    "style": false,
    "font": "",
    "fontSize": 16,
    "color": "#717171",
    "wrapText": true,
    "className": "",
    "value": "payload.P1",
    "valueType": "msg",
    "x": 1700,
    "y": 520,
    "wires": []
  },
  {
    "id": "5e1c31f6c108fa2d",
    "type": "http in",
    "z": "72b54a41227aca2a",
    "name": "Get Scores API",
    "url": "/api/scores",
    "method": "get",
    "upload": false,
    "skipBodyParsing": false,
    "swaggerDoc": "",
    "x": 240,
    "y": 1160,
    "wires": [
      [
        "757966c62152c55a"
      ]
    ]
  },
  {
    "id": "757966c62152c55a",
    "type": "function",
    "z": "72b54a41227aca2a",
    "name": "Get Scores",
```

```

        "func": "let scores = flow.get(\"scores\") ||
{\n    P1: 0,\n    P2: 0,\n    P3: 0\n};\n\nmsg.payload =
{\n    success: true,\n    scores: scores,\n    roundLocked:
flow.get(\"roundLocked\") || false\n};\n\nreturn msg;",
        "outputs": 1,
        "timeout": 0,
        "noerr": 0,
        "initialize": "",
        "finalize": "",
        "libs": [],
        "x": 730,
        "y": 1160,
        "wires": [
            [
                "91862a080456440e"
            ]
        ]
    },
    {
        "id": "91862a080456440e",
        "type": "http response",
        "z": "72b54a41227aca2a",
        "name": "",
        "statusCode": "",
        "headers": {},
        "x": 1230,
        "y": 1160,
        "wires": []
    },
    {
        "id": "425398f68078983f",
        "type": "function",
        "z": "72b54a41227aca2a",
        "name": "Reset All Scores",
        "func": "let scores = {\n    P1: 0,\n    P2:
0,\n    P3: 0\n};\n\nflow.set(\"scores\",
scores);\n\nmsg.payload = scores;\nreturn msg;",

```

```
    "outputs": 1,  
    "timeout": 0,  
    "noerr": 0,  
    "initialize": "",  
    "finalize": "",  
    "libs": [],  
    "x": 940,  
    "y": 760,  
    "wires": [  
      [  
        "21b38363a605f9a3",  
        "190b2c798d01e8fe",  
        "6569064ea7fa08d9",  
        "db21b37b840bbb57"  
      ]  
    ]  
  },  
  {  
    "id": "6265cfdc670b9efd",  
    "type": "inject",  
    "z": "72b54a41227aca2a",  
    "name": "Reset Scores Test",  
    "props": [  
      {  
        "p": "payload"  
      },  
      {  
        "p": "topic",  
        "vt": "str"  
      }  
    ],  
    "repeat": "",  
    "crontab": "",  
    "once": false,  
    "onceDelay": 0.1,  
    "topic": "",  
    "payload": "",
```

```
    "payloadType": "date",
    "x": 270,
    "y": 760,
    "wires": [
      [
        "a82e2f4782b70879"
      ]
    ]
  },
  {
    "id": "f563ba1175c3d65c",
    "type": "ui-text",
    "z": "72b54a41227aca2a",
    "group": "0851430a4244eecb",
    "order": 11,
    "width": 0,
    "height": 0,
    "name": "Round Status",
    "label": "",
    "format": "{{msg.payload}}",
    "layout": "row-left",
    "style": false,
    "font": "",
    "fontSize": 16,
    "color": "#717171",
    "wrapText": false,
    "className": "",
    "value": "payload",
    "valueType": "msg",
    "x": 1220,
    "y": 460,
    "wires": []
  },
  {
    "id": "9c2bbb531a6be0c6",
    "type": "function",
    "z": "72b54a41227aca2a",
```

```

        "name": "Round Locked Status",
        "func": "msg.payload = \" الجولة مقفلة - تم تسجيل أول لاعب\"",
    };
    return msg;
    "outputs": 1,
    "timeout": 0,
    "noerr": 0,
    "initialize": "",
    "finalize": "",
    "libs": [],
    "x": 980,
    "y": 420,
    "wires": [
        [
            "f563ba1175c3d65c"
        ]
    ]
},
{
    "id": "97e14cca3ad9a8e0",
    "type": "function",
    "z": "72b54a41227aca2a",
    "name": "Round Ready Status",
    "func": "msg.payload = \" الجولة جاهزة - يمكن للاعبين الضغط\"",
    };
    return msg;
    "outputs": 1,
    "timeout": 0,
    "noerr": 0,
    "initialize": "",
    "finalize": "",
    "libs": [],
    "x": 980,
    "y": 500,
    "wires": [
        [
            "f563ba1175c3d65c"
        ]
    ]
}

```

```
},  
{  
  "id": "d9a4bf51ae5e72ae",  
  "type": "ui-button",  
  "z": "72b54a41227aca2a",  
  "group": "0851430a4244eecb",  
  "name": "Dashboard Reset Round",  
  "label": "بدء جولة جديدة",  
  "order": 12,  
  "width": 0,  
  "height": 0,  
  "emulateClick": false,  
  "tooltip": "",  
  "color": "",  
  "bgcolor": "",  
  "className": "",  
  "icon": "",  
  "iconPosition": "left",  
  "payload": "03",  
  "payloadType": "str",  
  "topic": "topic",  
  "topicType": "msg",  
  "buttonColor": "",  
  "textColor": "",  
  "iconColor": "",  
  "enableClick": true,  
  "enablePointerdown": false,  
  "pointerdownPayload": "",  
  "pointerdownPayloadType": "str",  
  "enablePointerup": false,  
  "pointerupPayload": "",  
  "pointerupPayloadType": "str",  
  "x": 270,  
  "y": 500,  
  "wires": [  
    [  
      "eda118db274e4d79"
```

```

    ]
  ]
},
{
  "id": "a82e2f4782b70879",
  "type": "ui-button",
  "z": "72b54a41227aca2a",
  "group": "0851430a4244eecb",
  "name": "Dashboard Reset Scores",
  "label": "تصفير النتائج",
  "order": 13,
  "width": 0,
  "height": 0,
  "emulateClick": false,
  "tooltip": "",
  "color": "",
  "bgcolor": "",
  "className": "",
  "icon": "",
  "iconPosition": "left",
  "payload": "reset_scores",
  "payloadType": "str",
  "topic": "topic",
  "topicType": "msg",
  "buttonColor": "",
  "textColor": "",
  "iconColor": "",
  "enableClick": true,
  "enablePointerdown": false,
  "pointerdownPayload": "",
  "pointerdownPayloadType": "str",
  "enablePointerup": false,
  "pointerupPayload": "",
  "pointerupPayloadType": "str",
  "x": 590,
  "y": 760,
  "wires": [

```

```

        [
            "425398f68078983f"
        ]
    ],
    {
        "id": "8028a1b6382dd09f",
        "type": "http in",
        "z": "72b54a41227aca2a",
        "name": "Reset Round API",
        "url": "/api/reset-round",
        "method": "post",
        "upload": false,
        "skipBodyParsing": false,
        "swaggerDoc": "",
        "x": 240,
        "y": 640,
        "wires": [
            [
                "e2e557cf1263a91b"
            ]
        ]
    },
    {
        "id": "e2e557cf1263a91b",
        "type": "function",
        "z": "72b54a41227aca2a",
        "name": "API Reset Round",
        "func": "flow.set(\"roundLocked\",
false);\n\nmsg.payload = {\n    success: true,\n    message:
\"Round reset successfully\", \n    // حالة الجولة غير
مقفولة\n    roundLocked: false\n};\n\nreturn msg;",
        "outputs": 1,
        "timeout": 0,
        "noerr": 0,
        "initialize": "",
        "finalize": "",

```

```
    "libs": [],
    "x": 590,
    "y": 640,
    "wires": [
      [
        "e052088121fe5f35",
        "d9fe473f0dd02692"
      ]
    ]
  },
  {
    "id": "d9fe473f0dd02692",
    "type": "http response",
    "z": "72b54a41227aca2a",
    "name": "",
    "statusCode": "",
    "headers": {},
    "x": 930,
    "y": 680,
    "wires": []
  },
  {
    "id": "61e7867d2c2949f5",
    "type": "debug",
    "z": "72b54a41227aca2a",
    "name": "debug 34",
    "active": true,
    "tosidebar": true,
    "console": false,
    "tostatus": false,
    "complete": "false",
    "statusVal": "",
    "statusType": "auto",
    "x": 420,
    "y": 100,
    "wires": []
  },
}
```

```
{
  "id": "e9d05f69d9f5670c",
  "type": "debug",
  "z": "72b54a41227aca2a",
  "name": "debug 35",
  "active": true,
  "tosidebar": true,
  "console": false,
  "tostatus": false,
  "complete": "payload",
  "targetType": "msg",
  "statusVal": "",
  "statusType": "auto",
  "x": 700,
  "y": 140,
  "wires": []
},
{
  "id": "21b38363a605f9a3",
  "type": "debug",
  "z": "72b54a41227aca2a",
  "name": "debug 36",
  "active": true,
  "tosidebar": true,
  "console": false,
  "tostatus": false,
  "complete": "true",
  "targetType": "full",
  "statusVal": "",
  "statusType": "auto",
  "x": 1200,
  "y": 760,
  "wires": []
},
{
  "id": "6569064ea7fa08d9",
  "type": "ui-text",
```

```
    "z": "72b54a41227aca2a",
    "group": "0851430a4244eecb",
    "order": 9,
    "width": 0,
    "height": 0,
    "name": "Player 2 Score",
    "label": "Player 2 Score",
    "format": "{{msg.payload}}",
    "layout": "row-spread",
    "style": false,
    "font": "",
    "fontSize": 16,
    "color": "#717171",
    "wrapText": true,
    "className": "",
    "value": "payload.P2",
    "valueType": "msg",
    "x": 1700,
    "y": 580,
    "wires": []
  },
  {
    "id": "db21b37b840bbb57",
    "type": "ui-text",
    "z": "72b54a41227aca2a",
    "group": "0851430a4244eecb",
    "order": 10,
    "width": 0,
    "height": 0,
    "name": "Player 3 Score",
    "label": "Player 3 Score",
    "format": "{{msg.payload}}",
    "layout": "row-spread",
    "style": false,
    "font": "",
    "fontSize": 16,
    "color": "#717171",
```

```
    "wrapText": true,  
    "className": "",  
    "value": "payload.P3",  
    "valueType": "msg",  
    "x": 1700,  
    "y": 640,  
    "wires": []  
  },  
  {  
    "id": "9beb1883642ad027",  
    "type": "ui-text",  
    "z": "72b54a41227aca2a",  
    "group": "0851430a4244eecb",  
    "order": 7,  
    "width": 0,  
    "height": 0,  
    "name": "First Player Display",  
    "label": "",  
    "format": "{{msg.payload}}",  
    "layout": "row-left",  
    "style": false,  
    "font": "",  
    "fontSize": 16,  
    "color": "#717171",  
    "wrapText": true,  
    "className": "",  
    "value": "payload",  
    "valueType": "msg",  
    "x": 970,  
    "y": 240,  
    "wires": []  
  },  
  {  
    "id": "11d47a9251596f78",  
    "type": "function",  
    "z": "72b54a41227aca2a",  
    "name": "Questions",
```

```

"func": "let questions = [\n\n    // السؤال
1\n    {\n        q: \"السؤال الأول ما هو نظام التشغيل مفتوح \"\n        A: \"Windows\", \n        B: \n        C: \"iOS\", \n        D: \n        correct: \"D\" \n    }, \n\n    // السؤال
2\n    {\n        q: \"السؤال الثاني أي لغة برمجة تستخدم لتطوير تطبيقات \"\n        A: \"Python\", \n        B: \n        C: \"Java\", \n        D: \n        correct: \"C\" \n    }, \n\n    // السؤال
3\n    {\n        q: \"السؤال الثالث ما هي لغة ترميز النصوص \"\n        A: \"HTML\", \n        B: \n        C: \"CSS\", \n        D: \n        correct: \"A\" \n    }, \n\n    // السؤال
4\n    {\n        q: \"السؤال الرابع أي من هذه ليست لغة \"\n        A: \"Python\", \n        B: \n        C: \"HTML\", \n        D: \n        correct: \"C\" \n    }, \n\n    // السؤال
5\n    {\n        q: \"السؤال الخامس ما هو بروتوكول نقل النص \"\n        A: \"FTP\", \n        B: \n        C: \"SMTP\", \n        D: \n        correct: \"B\" \n    }, \n\n    // السؤال
6\n    {\n        q: \"السؤال السادس أي من هذه قواعد بيانات \"\n        A: \"MongoDB\", \n        B: \n        C: \"Redis\", \n        D: \n        correct: \"B\" \n    }, \n\n    // السؤال
7\n    {\n        q: \"المعروف باسم IP السؤال السابع ما هو الـ \"\n        A: \"192.168.1.1\", \n        B: \n        C: \"127.0.0.1\", \n        D: \n        correct: \"C\" \n    }, \n\n    //
8\n    {\n        q: \"السؤال الثامن أي من هذه ليس نظام تشغيل \"\n        A: \"iOS\", \n        B: \n        C: \"Windows Phone\", \n        D: \n        correct: \"D\" \n    }, \n\n    //
9\n    {\n        q: \"السؤال التاسع تستخدم لغة الـ \"\n        A: \"تصميم الواجهات\", \n        B: \"البرمجة \"\n        C: \"قواعد البيانات\", \n        D: \"الذكاء \"\n        correct: \"A\" \n    }, \n\n    // السؤال
الاصطناعي\n

```

```

10\n    {\n        q: \" السؤال العاشر أي من هذه ليس لغة برمجة \"
    نصية\", \n        A: \"JavaScript\", \n        B:
    \"Python\", \n        C: \"PHP\", \n        D:
    \"C#\", \n        correct: \"D\" \n    }, \n \n    // السؤال
11\n    {\n        q: \" السؤال الحادي عشر ما هو بروتوكول البريد \"
    الإلكتروني\", \n        A: \"SMTP\", \n        B:
    \"HTTP\", \n        C: \"FTP\", \n        D:
    \"TCP\", \n        correct: \"A\" \n    }, \n \n    // السؤال
12\n    {\n        q: \" السؤال الثاني عشر أي من هذه هو إطار عمل جافا \"
    سكريبت\", \n        A: \"React\", \n        B:
    \"Django\", \n        C: \"Laravel\", \n        D:
    \"Spring\", \n        correct: \"A\" \n    }, \n \n    // السؤال
13\n    {\n        q: \" السؤال الثالث عشر ما هي وحدة المعالجة \"
    المركزية\", \n        A: \"الذاكرة\", \n        B: \" القرص \"
    الصلب\", \n        C: \"المعالج\", \n        D: \" كارت \"
    الشاشة\", \n        correct: \"C\" \n    }, \n \n    // السؤال
14\n    {\n        q: \" السؤال الرابع عشر أي من هذه ليس نوع من \"
    الذاكرة\", \n        A: \"RAM\", \n        B:
    \"ROM\", \n        C: \"CPU\", \n        D:
    \"Cache\", \n        correct: \"C\" \n    }, \n \n    // السؤال
15\n    {\n        q: \" السؤال الخامس عشر ما هو الـ \"
    DNS؟\", \n        A: \" خادم أسماء النطاقات\", \n        B: \" بروتوكول \"
    نقل\", \n        C: \" لغة برمجة\", \n        D: \" نظام \"
    تشغيل\", \n        correct: \"A\" \n    }, \n \n    // السؤال
16\n    {\n        q: \" السؤال السادس عشر أي من هذه يستخدم للتحكم \"
    بالإصدارات\", \n        A: \"Git\", \n        B:
    \"Docker\", \n        C: \"Kubernetes\", \n        D:
    \"Jenkins\", \n        correct: \"A\" \n    }, \n \n    // السؤال
17\n    {\n        q: \" السؤال السابع عشر ما هو نظام التشغيل من \"
    Apple؟\", \n        A: \"Windows\", \n        B:
    \"Linux\", \n        C: \"macOS\", \n        D:
    \"Android\", \n        correct: \"C\" \n    }, \n \n    // السؤال
18\n    {\n        q: \" السؤال الثامن عشر أي من هذه ليس نوع من قواعد \"
    البيانات\", \n        A: \" علائقية\", \n        B:
    \" وثائقية\", \n        C: \" بيانية\", \n        D:
    \" خطية\", \n        correct: \"D\" \n    }, \n \n    // السؤال
19\n    {\n        q: \" السؤال التاسع عشر ما هو الـ \"
    API؟\", \n        A:

```

[illegible]

```

        "timeout": 0,
        "noerr": 0,
        "initialize": "",
        "finalize": "",
        "libs": [],
        "x": 1020,
        "y": 960,
        "wires": [
            [
                "c434f115778f34b7",
                "8cbc263ecedffdc9"
            ]
        ]
    },
    {
        "id": "037c7a51d569bc2d",
        "type": "function",
        "z": "72b54a41227aca2a",
        "name": "Check Answer",
        "func": "let answer =
String(msg.payload).trim();\n\nlet correctAnswer =
flow.get(\"correctAnswer\");\n\nif (answer ===
correctAnswer) {\n\n    msg.payload = \"✓ الإجابة صحيحة\";\n\n}
else {\n\n    msg.payload = \"✗ الإجابة خاطئة\";\n\n}\n\nreturn
[msg, null];",
        "outputs": 2,
        "timeout": 0,
        "noerr": 0,
        "initialize": "",
        "finalize": "",
        "libs": [],
        "x": 560,
        "y": 920,
        "wires": [
            [
                "b26cdedfdb493a01",
                "73721bf4264b2f68"
            ]
        ]
    }
]

```

```

        ],
        [
            "11d47a9251596f78"
        ]
    ]
},
{
    "id": "71d4ca747fd5624b",
    "type": "ui-button",
    "z": "72b54a41227aca2a",
    "group": "0851430a4244eecb",
    "name": "Answer A",
    "label": "A",
    "order": 2,
    "width": 0,
    "height": 0,
    "emulateClick": false,
    "tooltip": "",
    "color": "",
    "bgcolor": "",
    "className": "",
    "icon": "",
    "iconPosition": "left",
    "payload": "A",
    "payloadType": "str",
    "topic": "topic",
    "topicType": "msg",
    "buttonColor": "",
    "textColor": "",
    "iconColor": "",
    "enableClick": true,
    "enablePointerdown": false,
    "pointerdownPayload": "",
    "pointerdownPayloadType": "str",
    "enablePointerup": false,
    "pointerupPayload": "",
    "pointerupPayloadType": "str",

```

```
    "x": 220,  
    "y": 840,  
    "wires": [  
      [  
        "037c7a51d569bc2d"  
      ]  
    ]  
  },  
  {  
    "id": "86f5fcd93ffc34b2",  
    "type": "ui-button",  
    "z": "72b54a41227aca2a",  
    "group": "0851430a4244eecb",  
    "name": "Answer B",  
    "label": "B",  
    "order": 3,  
    "width": 0,  
    "height": 0,  
    "emulateClick": false,  
    "tooltip": "",  
    "color": "",  
    "bgcolor": "",  
    "className": "",  
    "icon": "",  
    "iconPosition": "left",  
    "payload": "B",  
    "payloadType": "str",  
    "topic": "topic",  
    "topicType": "msg",  
    "buttonColor": "",  
    "textColor": "",  
    "iconColor": "",  
    "enableClick": true,  
    "enablePointerdown": false,  
    "pointerdownPayload": "",  
    "pointerdownPayloadType": "str",  
    "enablePointerup": false,
```

```
    "pointerupPayload": "",
    "pointerupPayloadType": "str",
    "x": 220,
    "y": 900,
    "wires": [
      [
        "037c7a51d569bc2d"
      ]
    ]
  },
  {
    "id": "1c46a63a551b31b0",
    "type": "ui-button",
    "z": "72b54a41227aca2a",
    "group": "0851430a4244eecb",
    "name": "Answer C",
    "label": "C",
    "order": 4,
    "width": 0,
    "height": 0,
    "emulateClick": false,
    "tooltip": "",
    "color": "",
    "bgcolor": "",
    "className": "",
    "icon": "",
    "iconPosition": "left",
    "payload": "C",
    "payloadType": "str",
    "topic": "topic",
    "topicType": "msg",
    "buttonColor": "",
    "textColor": "",
    "iconColor": "",
    "enableClick": true,
    "enablePointerdown": false,
    "pointerdownPayload": "",
```

```
    "pointerdownPayloadType": "str",
    "enablePointerup": false,
    "pointerupPayload": "",
    "pointerupPayloadType": "str",
    "x": 220,
    "y": 960,
    "wires": [
      [
        "037c7a51d569bc2d"
      ]
    ]
  },
  {
    "id": "5fcdd2d2979ae531",
    "type": "ui-button",
    "z": "72b54a41227aca2a",
    "group": "0851430a4244eecb",
    "name": "Answer D",
    "label": "D",
    "order": 5,
    "width": 0,
    "height": 0,
    "emulateClick": false,
    "tooltip": "",
    "color": "",
    "bgcolor": "",
    "className": "",
    "icon": "",
    "iconPosition": "left",
    "payload": "D",
    "payloadType": "str",
    "topic": "topic",
    "topicType": "msg",
    "buttonColor": "",
    "textColor": "",
    "iconColor": "",
    "enableClick": true,
```

```
    "enablePointerdown": false,  
    "pointerdownPayload": "",  
    "pointerdownPayloadType": "str",  
    "enablePointerup": false,  
    "pointerupPayload": "",  
    "pointerupPayloadType": "str",  
    "x": 220,  
    "y": 1020,  
    "wires": [  
      [  
        "037c7a51d569bc2d"  
      ]  
    ]  
  },  
  {  
    "id": "c434f115778f34b7",  
    "type": "ui-text",  
    "z": "72b54a41227aca2a",  
    "group": "0851430a4244eecb",  
    "order": 1,  
    "width": 0,  
    "height": 0,  
    "name": "Question Display",  
    "label": "",  
    "format": "{{msg.payload}}",  
    "layout": "row-spread",  
    "style": false,  
    "font": "",  
    "fontSize": 16,  
    "color": "#717171",  
    "wrapText": true,  
    "className": "",  
    "value": "payload",  
    "valueType": "msg",  
    "x": 1270,  
    "y": 920,  
    "wires": []  
  }  
]
```

```
},
{
  "id": "db241eda5b2ad3d6",
  "type": "inject",
  "z": "72b54a41227aca2a",
  "name": "Start Questions",
  "props": [
    {
      "p": "payload"
    },
    {
      "p": "topic",
      "vt": "str"
    }
  ],
  "repeat": "",
  "crontab": "",
  "once": false,
  "onceDelay": 0.1,
  "topic": "",
  "payload": "START",
  "payloadType": "str",
  "x": 580,
  "y": 820,
  "wires": [
    [
      "b26cdedfdb493a01",
      "11d47a9251596f78"
    ]
  ]
},
{
  "id": "b26cdedfdb493a01",
  "type": "ui-text",
  "z": "72b54a41227aca2a",
  "group": "0851430a4244eecb",
  "order": 6,
```

```
    "width": 0,  
    "height": 0,  
    "name": "Answer Result",  
    "label": "",  
    "format": "{{msg.payload}}",  
    "layout": "row-spread",  
    "style": false,  
    "font": "",  
    "fontSize": 16,  
    "color": "#717171",  
    "wrapText": false,  
    "className": "",  
    "value": "payload",  
    "valueType": "msg",  
    "x": 940,  
    "y": 820,  
    "wires": []  
  },  
  {  
    "id": "73721bf4264b2f68",  
    "type": "debug",  
    "z": "72b54a41227aca2a",  
    "name": "debug 37",  
    "active": true,  
    "tosidebar": true,  
    "console": false,  
    "tostatus": false,  
    "complete": "payload",  
    "targetType": "msg",  
    "statusVal": "",  
    "statusType": "auto",  
    "x": 820,  
    "y": 980,  
    "wires": []  
  },  
  {  
    "id": "47883a09cf364adc",
```

```
    "type": "function",
    "z": "72b54a41227aca2a",
    "name": "Clear Answer Result",
    "func": "msg.payload = \"\";\nreturn msg;",
    "outputs": 1,
    "timeout": 0,
    "noerr": 0,
    "initialize": "",
    "finalize": "",
    "libs": [],
    "x": 1720,
    "y": 1020,
    "wires": [
      [
        "11d47a9251596f78",
        "b26cdedfdb493a01"
      ]
    ]
  },
  {
    "id": "8cbc263ecedffdc9",
    "type": "trigger",
    "z": "72b54a41227aca2a",
    "name": "Question Timer 20s",
    "op1": "",
    "op2": "NEXT",
    "op1type": "nul",
    "op2type": "str",
    "duration": "20",
    "extend": false,
    "overrideDelay": false,
    "units": "s",
    "reset": "",
    "bytopic": "all",
    "topic": "topic",
    "outputs": 1,
    "x": 1250,
```

```
    "y": 1060,
    "wires": [
      [
        "fcaaf035963c6e25",
        "b26cdedfdb493a01"
      ]
    ]
  },
  {
    "id": "fcaaf035963c6e25",
    "type": "function",
    "z": "72b54a41227aca2a",
    "name": "Next Question",
    "func": "let index = flow.get(\"questionIndex\") ||
0;\nindex++;\nflow.set(\"questionIndex\",
index);\n\nmsg.payload = \"NEXT\";\nreturn msg;",
    "outputs": 2,
    "timeout": 0,
    "noerr": 0,
    "initialize": "",
    "finalize": "",
    "libs": [],
    "x": 1480,
    "y": 1060,
    "wires": [
      [
        "47883a09cf364adc"
      ],
      [
        "53ec22c36cc12989"
      ]
    ]
  },
  {
    "id": "53ec22c36cc12989",
    "type": "debug",
    "z": "72b54a41227aca2a",
```

```
    "name": "debug 38",
    "active": true,
    "tosidebar": true,
    "console": false,
    "tostatus": false,
    "complete": "payload",
    "targetType": "msg",
    "statusVal": "",
    "statusType": "auto",
    "x": 1680,
    "y": 1120,
    "wires": []
  },
  {
    "id": "c4575839817bb6d1",
    "type": "mqtt-broker",
    "name": "Quiz MQTT Broker",
    "broker": "127.0.0.1",
    "port": 1883,
    "clientId": "",
    "autoConnect": true,
    "usetls": false,
    "protocolVersion": 4,
    "keepalive": 60,
    "cleansession": true,
    "autoUnsubscribe": true,
    "birthTopic": "",
    "birthQos": "0",
    "birthRetain": "false",
    "birthPayload": "",
    "birthMsg": {},
    "closeTopic": "",
    "closeQos": "0",
    "closeRetain": "false",
    "closePayload": "",
    "closeMsg": {},
    "willTopic": "",
```

```

    "willQos": "0",
    "willRetain": "false",
    "willPayload": "",
    "willMsg": {},
    "userProps": "",
    "sessionExpiry": ""
  },
  {
    "id": "0851430a4244eecb",
    "type": "ui-group",
    "name": "Quiz Game",
    "page": "45989f2a9f068c07",
    "width": 6,
    "height": 1,
    "order": 1,
    "showTitle": true,
    "className": "",
    "visible": "true",
    "disabled": "false",
    "groupType": "default"
  },
  {
    "id": "45989f2a9f068c07",
    "type": "ui-page",
    "name": "Quiz Game",
    "ui": "484a7449355702a4",
    "path": "/quiz",
    "icon": "home",
    "layout": "grid",
    "theme": "3b10a9b45f851c48",
    "breakpoints": [
      {
        "name": "Default",
        "px": "0",
        "cols": "3"
      },
      {

```

```
        "name": "Tablet",
        "px": "576",
        "cols": "6"
    },
    {
        "name": "Small Desktop",
        "px": "768",
        "cols": "9"
    },
    {
        "name": "Desktop",
        "px": "1024",
        "cols": "12"
    }
],
"order": 1,
"className": "",
"visible": "true",
"disabled": "false"
},
{
    "id": "484a7449355702a4",
    "type": "ui-base",
    "name": "My Dashboard",
    "path": "/dashboard",
    "appIcon": "",
    "includeClientData": true,
    "acceptsClientConfig": [
        "ui-notification",
        "ui-control"
    ],
    "showPathInSidebar": false,
    "headerContent": "dashpage",
    "navigationStyle": "default",
    "titleBarStyle": "fixed",
    "showReconnectNotification": true,
    "notificationDisplayTime": 1,
```

```
    "showDisconnectNotification": true,  
    "allowInstall": true  
  },  
  {  
    "id": "3b10a9b45f851c48",  
    "type": "ui-theme",  
    "name": "Default Theme",  
    "colors": {  
      "surface": "#976a9a",  
      "primary": "#d28ecc",  
      "bgPage": "#f3d1f5",  
      "groupBg": "#dfb6e2",  
      "groupOutline": "#f2d2f9"  
    },  
    "sizes": {  
      "density": "default",  
      "pagePadding": "12px",  
      "groupGap": "12px",  
      "groupBorderRadius": "4px",  
      "widgetGap": "12px"  
    }  
  },  
  {  
    "id": "64cdcd638ec0cc4b",  
    "type": "global-config",  
    "env": [],  
    "modules": {  
      "@flowfuse/node-red-dashboard": "1.30.2"  
    }  
  }  
}
```

]

Appendix B: Custom CSS / HTML Snippets (Dashboard Template Nodes)

No custom CSS or HTML snippets were implemented using Dashboard Template nodes in the current project.

Appendix C: Team Contribution

The project was developed collaboratively by all team members. The team members participated in the implementation, integration, testing, debugging, and documentation of the IoT-based quiz game.

Team Contribution Table

Team Member	Contribution / Tasks Performed
Maisoon Mansoor	Hardware + ESP32 Programming → MQTT → Node-RED Game Logic → Dashboard 2.0 → REST API/Flutter → Testing & Documentation.
Bashayer Mansoor	Hardware + ESP32 Programming → MQTT → Node-RED Game Logic → Dashboard 2.0 → REST API/Flutter → Testing & Documentation.
Zainab Mansoor	Hardware + ESP32 Programming → MQTT → Node-RED Game Logic → Dashboard 2.0 → REST API/Flutter → Testing & Documentation.
Shahad Ali	Hardware + ESP32 Programming → MQTT → Node-RED Game Logic → Dashboard 2.0 → REST API/Flutter → Testing & Documentation.
Alia'a Abdullah	Hardware + ESP32 Programming → MQTT → Node-RED Game Logic → Dashboard 2.0 → REST API/Flutter → Testing & Documentation.

End of template